



PYTHON

VIBE CODING

COURSE SLIDES · WSQ

Build and Deploy Python Applications with Vibe Coding

WSQ Course Code: TGS-2019504591

Conducted by Tertiary Infotech Academy Pte Ltd · UEN 201200696W

Version v1.5 · 20 July 2026

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg



COURSE ADMINISTRATION

Welcome & Housekeeping

Digital Attendance (Mandatory)

- It is mandatory to take the AM, PM and Assessment digital attendance for WSQ-funded courses.
- The trainer/administrator displays the digital attendance QR code from the SSG portal.
- Scan the QR code with your mobile phone camera and submit your attendance.
- A minimum of 75% attendance is required to be eligible for assessment and funding.

About the Trainer



Your Trainer

General Trainer template —
to be completed by the trainer

NAME

TITLE / DESIGNATION

QUALIFICATIONS

AREAS OF EXPERTISE

TRAINING & INDUSTRY EXPERIENCE

CONTACT

About the Trainer



Dr. Alfred Ang

Principal Trainer
Tertiary Infotech Academy Pte. Ltd.

ROLE

Principal Trainer, Tertiary Infotech Academy Pte. Ltd.

EXPERTISE

Python application development, AI-assisted (vibe) coding, data analytics and deployment.

DELIVERS

WSQ courses on Python, AI-assisted development, data engineering & software delivery.

FOUNDER

Founder and lead instructor at Tertiary Infotech / Tertiary Courses.

Let's Know Each Other

- Your name and organisation / role.
- Your experience with Python and with AI coding assistants (if any).
- What you want to build and deploy after this course.

Ground Rules

1

Set your mobile phone to silent mode.

2

Participate actively — no question is too small.

3

Mutual respect: agree to disagree.

4

One conversation at a time.

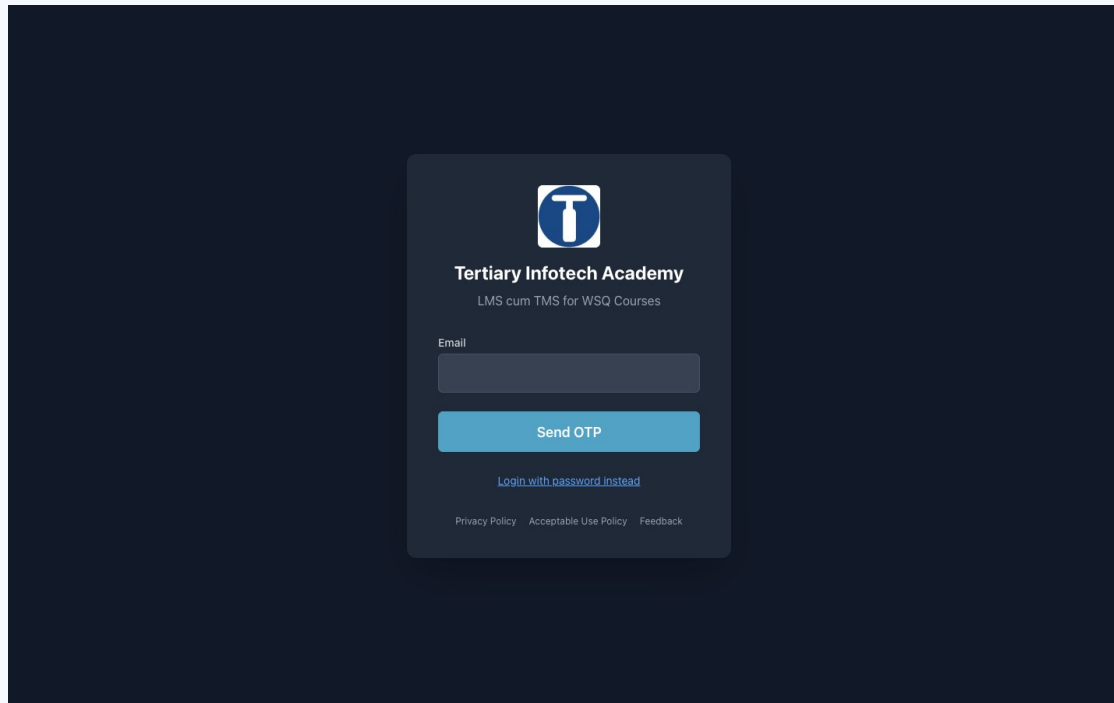
5

Be punctual; return from breaks on time.

6

75% attendance is required.

Download Your Course Material



- Sign in with the email address you used to register for this course.
- Download the slides and the Learner Guide before the assessment.
- The assessment is OPEN BOOK — have both open on the day.
- Your attendance and assessment results are recorded here too.

<https://lms-tms.tertiaryinfotech.com>

SCHEDULE

Lesson Plan — 3 Days, 8 hours/day

Days 1–2

- Day 1 — Vibe Coding Foundations & Object-Oriented Python
 - Topic 1: Vibe Coding Foundations (Labs 1–4)
 - Topic 2: Object-Oriented Programming in Python (Labs 5–8)
- Day 2 — Data Analytics with pandas, Pydantic & FastAPI
 - Topic 3: Data Analytics with pandas (Labs 9–12)
 - Topic 4: Data Modelling with Pydantic and FastAPI begins (Labs 13–16)

Day 3 & timing

- Day 3 — FastAPI, Packaging, Deployment & Assessment
 - Topic 4: Data Modelling with Pydantic and FastAPI completes
 - Topic 5: Packaging and Deployment (Labs 17–20)
 - Final Assessment (WA + PP)
- Daily timing
 - 9:30am–6:30pm · 1-hour lunch · tea breaks within

Learning Outcomes

1

LO1

Apply vibe coding practices — set up a reproducible Python project with uv, and use an AI coding assistant to scaffold, explain and refactor working code.

2

LO2

Design and implement object-oriented Python — classes, encapsulation, inheritance and composition — using AI-assisted conversational design.

3

LO3

Build data analytics pipelines with pandas — load, clean, transform, group and aggregate realistic datasets into reportable results.

4

LO4

Model and validate data with Pydantic and expose it through a FastAPI application with typed request and response models.

5

LO5

Package and deploy a Python application — dependency locking, configuration, containerisation and a running deployed service.

Briefing for Assessment

1

Clear your desk

Phones and materials go under the table or on the floor.

2

No photos or recording

Assessment scripts must not be photographed or recorded.

3

No discussion

Work individually — no talking during the assessment.

4

Black or blue pen

Use a black/blue pen for hard-copy assessments.

5

No correction fluid

Liquid paper and correction tape are not permitted.

6

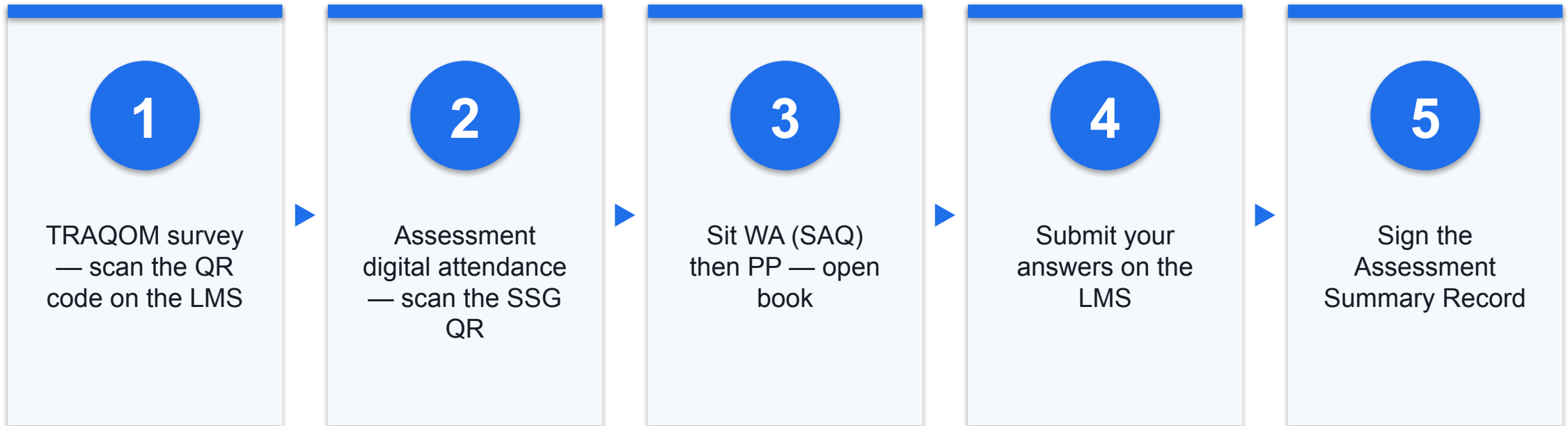
Scripts collected

All scripts are collected when time is up.

Assessment

- Written Assessment (WA) — Short-Answer Questions (SAQ), 1 hour, open book.
- Practical Performance (PP) — hands-on Python build-and-deploy tasks, 1 hour, open book.
- Format: Open Book — slides, Learner Guide and approved materials only.
- A minimum of 75% attendance is required to be eligible for assessment and funding.
- An appeal process is available if required.

Assessment Flow





CORE CONCEPTS

What Vibe Coding Is — and What It Is Not

What is Vibe Coding?

1

AI-assisted, human-owned

You specify, the assistant drafts, you review and own the result.

2

Specification over guesswork

State the goal, constraints, inputs and expected output.

3

Verify, never assume

Generated code that runs is not the same as generated code that is correct.

4

Faster iteration

Refactoring conversationally beats rewriting by hand — with a test to prove it.

The Vibe Coding Loop

1 · Specify

- State the goal precisely
- Name the constraints
- Give real inputs and expected output

2 · Generate

- Let the assistant draft
- Read every line before running
- Ask for the shape you want

3 · Verify

- Run it against edge cases
- Test the error paths
- Fix by naming the defect, not 'improve this'

WHY IT MATTERS

You are accountable for every line you ship.

The assistant drafts; you review, test and own the result. That is the difference between vibe coding and guessing.

The Toolchain for This Course

Build

- uv
 - Project, interpreter and locked dependencies
 - uv init · uv add · uv run
- pandas 3.0
 - DataFrames, cleaning, groupby, rolling windows
 - Copy-on-Write is the default

Serve & ship

- Pydantic + FastAPI
 - Typed models, validation, HTTP endpoints
 - OpenAPI docs for free
- Streamlit + Docker
 - Analyst UI and a deployable image
 - One image, any machine

Your Workbench

- uv manages Python, the virtual environment and the lockfile — no venv activation by hand.
- An AI coding assistant (Claude, Copilot or Cursor) drafts code you review.
- SQLite ships with Python — the database needs no server and runs offline.
- Every lab in this course builds one application: CardGuard, a card-transaction fraud screening system.

TOPIC 01

Vibe Coding Foundations

uv Projects · AI Coding Assistants · Prompting Patterns · AI-Assisted Refactoring

Key Concepts — Vibe Coding Foundations

1

Vibe coding is directing an AI assistant to write code you specify, review and own — you remain accountable for correctness.

2

uv manages the interpreter, virtual environment and locked dependencies: `uv init`, `uv add`, `uv run`.

3

Effective prompts state the goal, the constraints, the inputs and the expected output — vague prompts produce plausible but wrong code.

4

Always read and run AI-generated code before trusting it; refactoring conversationally is faster than rewriting by hand.

Hands-On Labs — Vibe Coding Foundations

Labs 1–2

- Set Up a Reproducible Python Project with uv
- Write Effective Prompts for an AI Coding Assistant

Labs 3–4

- Review and Correct AI-Generated Code
- Refactor Working Code Conversationally

—

- —

Set Up a Reproducible Python Project with uv

LAB 1

The learner installs uv, creates a project, adds locked dependencies, and runs code through uv so the environment is identical on every machine. This is the foundation every later lab builds on.

You'll build

A uv project with pyproject.toml, uv.lock and a working entry point

Tools & libraries: uv, Python 3.12, pyproject.toml, uv.lock

LAB 1

Set Up a Reproducible Python Project with uv

1

STEP 1 OF 9

Confirm uv is installed and check the version

```
uv --version
```

WHAT THIS DOES

- Run this in your terminal, from the project folder.

Set Up a Reproducible Python Project with uv

2

STEP 2 OF 9

Create a new project folder and initialise it

```
uv init cardguard  
cd cardguard
```

WHAT THIS DOES

- Run this in your terminal, from the project folder.

Set Up a Reproducible Python Project with uv

3

STEP 3 OF 9

Inspect what uv generated — note pyproject.toml is the project manifest

```
ls -a  
cat pyproject.toml
```

WHAT THIS DOES

- Inspect what uv generated — note pyproject.toml is the project manifest

Set Up a Reproducible Python Project with uv

4

STEP 4 OF 9

Pin the Python version so every learner runs the same interpreter

```
uv python pin 3.12
```

WHAT THIS DOES

- Run this in your terminal, from the project folder.

Set Up a Reproducible Python Project with uv

5

STEP 5 OF 9

Add a dependency — uv resolves, installs and writes uv.lock in one step

```
uv add pandas
```

WHAT THIS DOES

- Run this in your terminal, from the project folder.
- uv resolves, installs and records the dependency in uv.lock.

Set Up a Reproducible Python Project with uv

6

STEP 6 OF 9

Open uv.lock and observe the exact pinned versions

```
head -30 uv.lock
```

WHAT THIS DOES

- Open uv.lock and observe the exact pinned versions

Set Up a Reproducible Python Project with uv

7

STEP 7 OF 9 · CONT. 1/2

Write a first script that proves the dependency is importable

```
# main.py
import pandas as pd

def main() -> None:
    df = pd.DataFrame({"card": ["4111...1111", "5500...0004",
                                "4111...1111"],
                       "merchant": ["SG Grocer", "Overseas
                                   ATM", "SG Grocer"],
                       "amount": [42.90, 800.00, 15.50]})
    print(df)
    print(f"Total screened: ${df['amount'].sum():,.2f}")

if __name__ == "__main__":
```

WHAT THIS DOES

- Read the signature first: what goes in, what comes out.

Set Up a Reproducible Python Project with uv

7

STEP 7 OF 9 · CONT. 2/2

Write a first script that proves the dependency is importable

```
main()
```

WHAT THIS DOES

- Read the signature first: what goes in, what comes out.

Set Up a Reproducible Python Project with uv

8

STEP 8 OF 9

Run it through uv — no venv activation needed

```
uv run python main.py
```

WHAT THIS DOES

- Run this in your terminal, from the project folder.
- uv run executes inside the project environment — no venv activation.

Set Up a Reproducible Python Project with uv

9

STEP 9 OF 9

Prove reproducibility: delete the venv and restore it from the lockfile

```
rm -rf .venv
uv sync
uv run python main.py
```

WHAT THIS DOES

- `uv run` executes inside the project environment — no venv activation.

Set Up a Reproducible Python Project with uv

Test it

`uv run python main.py` prints the transaction DataFrame and the screened total. After deleting `.venv`, `uv sync` restores it and the script produces identical output.

Write Effective Prompts for an AI Coding Assistant

LAB 2

The learner contrasts a vague prompt with a specified prompt on the same task, then applies the goal-constraints-inputs-output pattern to generate a function that handles real edge cases.

You'll build

Two versions of a transaction risk-scoring function and a written comparison of the prompts that produced them

Tools & libraries: AI coding assistant (Claude / Copilot / Cursor), uv, Python

Write Effective Prompts for an AI Coding Assistant

1

STEP 1 OF 7

Start from the vague prompt and record exactly what you get back

```
Ask the assistant: write a function to score a transaction
```

WHAT THIS DOES

- Start from the vague prompt and record exactly what you get back

Write Effective Prompts for an AI Coding Assistant

2

STEP 2 OF 7

Read the generated code critically — list what it assumed. Typical gaps: what counts as high value, whether an overseas merchant matters, what the score range is, what happens for a negative amount.

Write Effective Prompts for an AI Coding Assistant

3

STEP 3 OF 7

Rewrite the prompt using the four-part pattern — goal, constraints, inputs, expected output

```
Ask the assistant: Write a Python function
score_transaction(amount: float, is_overseas: bool, hour:
int) -> dict that returns a risk score from 0 to 100 and
the reasons that contributed to it. Add 40 points when
the amount exceeds $500, 30 points when the merchant is
overseas, and 20 points when the hour is between 1am and
5am. Cap the score at 100. Raise ValueError for a
negative amount or an hour outside 0-23. Return the score
and a list of reason strings.
```

WHAT THIS DOES

- Rewrite the prompt using the four-part pattern — goal, constraints, inputs, expected output

Write Effective Prompts for an AI Coding Assistant

4

STEP 4 OF 7 · CONT. 1/2

Save the improved result and read every line before running it

```
# scoring.py
def score_transaction(amount: float, is_overseas: bool, hour:
int) -> dict:
    """Return a 0-100 risk score for one transaction, with
    its reasons."""
    if amount < 0:
        raise ValueError("amount must not be negative")
    if not 0 <= hour <= 23:
        raise ValueError("hour must be between 0 and 23")

    HIGH_VALUE = 500.00
    score, reasons = 0, []
```

WHAT THIS DOES

- Read the signature first: what goes in, what comes out.
- The docstring states the contract the function promises.
- Invalid input fails fast and loudly, never silently.

Write Effective Prompts for an AI Coding Assistant

4

STEP 4 OF 7 · CONT. 2/2

Save the improved result and read every line before running it

```
if amount > HIGH_VALUE:
    score += 40
    reasons.append("high value")
if is_overseas:
    score += 30
    reasons.append("overseas merchant")
if 1 <= hour <= 5:
    score += 20
    reasons.append("unusual hour")

return {"score": min(score, 100), "reasons": reasons}
```

WHAT THIS DOES

- Read the signature first: what goes in, what comes out.
- The docstring states the contract the function promises.
- Invalid input fails fast and loudly, never silently.

Write Effective Prompts for an AI Coding Assistant

5

STEP 5 OF 7

Test the boundaries the prompt specified — this is where generated code usually fails

```
uv run python -c "  
from scoring import score_transaction  
print(score_transaction(42.90, False, 14)) # low risk  
print(score_transaction(800.00, True, 3)) # every rule  
    fires  
print(score_transaction(500.00, False, 14)) # exactly on the  
    threshold  
"
```

WHAT THIS DOES

- Run this in your terminal, from the project folder.
- `uv run` executes inside the project environment — no `venv` activation.

Write Effective Prompts for an AI Coding Assistant

6

STEP 6 OF 7

Confirm the error paths actually raise

```
uv run python -c "  
from scoring import score_transaction  
try:  
    score_transaction(-100, False, 14)  
except ValueError as e:  
    print('correctly raised:', e)  
"
```

WHAT THIS DOES

- Run this in your terminal, from the project folder.
- `uv run` executes inside the project environment — no `venv` activation.
- `try/except` handles the failure; finally always cleans up.

Write Effective Prompts for an AI Coding Assistant

7

STEP 7 OF 7

Write two sentences in your notes: which prompt produced code you would ship, and why.

Write Effective Prompts for an AI Coding Assistant

Test it

The specified prompt produces a function that caps the score at 100, fires each rule independently, and raises `ValueError` on invalid input. The learner can state why the vague prompt was insufficient.

Review and Correct AI-Generated Code

LAB 3

The learner is given a plausible but subtly wrong implementation, finds the defects by testing rather than by reading alone, and corrects them. This lab establishes that the developer, not the assistant, owns correctness.

You'll build

A corrected discount calculator plus a short defect log

Tools & libraries: AI coding assistant, uv, Python

Review and Correct AI-Generated Code

1

STEP 1 OF 7

Save this generated function exactly as written — it looks reasonable

```
# discount.py
def apply_discount(price, tier):
    if tier == "gold":
        return price * 0.8
    elif tier == "silver":
        return price * 0.9
    elif tier == "bronze":
        return price * 0.95
    return price
```

WHAT THIS DOES

- Read the signature first: what goes in, what comes out.

Review and Correct AI-Generated Code

2

STEP 2 OF 7

Probe it with the values a real order system would send

```
uv run python -c "  
from discount import apply_discount  
print(apply_discount(100, 'gold'))  
print(apply_discount(100, 'GOLD'))  
print(apply_discount(100, None))  
print(apply_discount(-50, 'gold'))  
print(apply_discount(100.555, 'silver'))  
"
```

WHAT THIS DOES

- Run this in your terminal, from the project folder.
- `uv run` executes inside the project environment — no venv activation.

Review and Correct AI-Generated Code

3

STEP 3 OF 7

Record the defects you found: case sensitivity, no validation of price, unrounded currency, silent fallthrough on an unknown tier, and no type hints.

Review and Correct AI-Generated Code

4

STEP 4 OF 7

Ask the assistant to fix the specific defects — name them, do not say 'improve this'

```
Ask the assistant: Fix these defects in apply_discount: (1) tier matching must be case-insensitive, (2) raise ValueError for a negative price, (3) raise ValueError for an unrecognised tier instead of silently returning the full price, (4) round the result to 2 decimal places, (5) add type hints and a docstring.
```

WHAT THIS DOES

- Invalid input fails fast and loudly, never silently.

Review and Correct AI-Generated Code

5

STEP 5 OF 7

Review the corrected version line by line before accepting it

```
# discount.py
DISCOUNTS = {"gold": 0.20, "silver": 0.10, "bronze": 0.05}

def apply_discount(price: float, tier: str) -> float:
    """Return the price after applying the tier discount."""
    if price < 0:
        raise ValueError("price must not be negative")
    key = (tier or "").strip().lower()
    if key not in DISCOUNTS:
        raise ValueError(f"unknown tier: {tier!r}")
    return round(price * (1 - DISCOUNTS[key]), 2)
```

WHAT THIS DOES

- Read the signature first: what goes in, what comes out.
- The docstring states the contract the function promises.
- Invalid input fails fast and loudly, never silently.

Review and Correct AI-Generated Code

6

STEP 6 OF 7

Re-run every probe that previously failed

```
uv run python -c "  
from discount import apply_discount  
print(apply_discount(100, 'GOLD'))  
print(apply_discount(100.555, 'silver'))  
for bad in [(-50, 'gold'), (100, 'platinum'), (100, None)]:  
    try:  
        apply_discount(*bad)  
        print('NOT CAUGHT:', bad)  
    except ValueError as e:  
        print('correctly raised:', e)  
"
```

WHAT THIS DOES

- Run this in your terminal, from the project folder.
- `uv run` executes inside the project environment — no `venv` activation.
- `try/except` handles the failure; finally always cleans up.

Review and Correct AI-Generated Code

7

STEP 7 OF 7

Note the lesson: the first version ran without error on the happy path. Running is not the same as correct.

Review and Correct AI-Generated Code

Test it

All five defects are fixed and demonstrated by tests. The learner can explain why reading alone did not reveal them.

Refactor Working Code Conversationally

LAB 4

The learner takes a working but poorly structured script, establishes a behavioural baseline, refactors it into small testable functions with AI assistance, and proves the output is unchanged.

You'll build

A refactored sales summary script with an unchanged, verified output

Tools & libraries: AI coding assistant, uv, Python

Refactor Working Code Conversationally

1

STEP 1 OF 7 · CONT. 1/2

Save the working script — it produces correct output but is one long block

```
# sales_report.py
orders = [
    {"id": 1, "region": "North", "amount": 1200.50, "status":
      "paid"},
    {"id": 2, "region": "South", "amount": 890.00, "status":
      "paid"},
    {"id": 3, "region": "North", "amount": 450.25, "status":
      "refunded"},
    {"id": 4, "region": "East", "amount": 2100.75, "status":
      "paid"},
    {"id": 5, "region": "South", "amount": 310.00, "status":
      "pending"},
]
```

WHAT THIS DOES

- Save the working script — it produces correct output but is one long block

Refactor Working Code Conversationally

1

STEP 1 OF 7 · CONT. 2/2

Save the working script — it produces correct output but is one long block

```
totals = {}
for o in orders:
    if o["status"] == "paid":
        if o["region"] not in totals:
            totals[o["region"]] = 0
        totals[o["region"]] += o["amount"]
for r in sorted(totals):
    print(f"{r}: ${totals[r]:,.2f}")
print(f"TOTAL: ${sum(totals.values()):,.2f}")
```

WHAT THIS DOES

- Save the working script — it produces correct output but is one long block

Refactor Working Code Conversationally

2

STEP 2 OF 7

Capture the baseline output to a file — this is the contract the refactor must preserve

```
uv run python sales_report.py > baseline.txt  
cat baseline.txt
```

WHAT THIS DOES

- Run this in your terminal, from the project folder.
- `uv run` executes inside the project environment — no `venv` activation.

Refactor Working Code Conversationally

3

STEP 3 OF 7

Ask for a structural refactor and state explicitly that behaviour must not change

```
Ask the assistant: Refactor sales_report.py into three
functions – filter_paid(orders), total_by_region(orders)
and format_report(totals) – plus a main() guarded by if
__name__ == '__main__'. Add type hints. The printed
output must be byte-for-byte identical to the current
version.
```

WHAT THIS DOES

- Ask for a structural refactor and state explicitly that behaviour must not change

Refactor Working Code Conversationally

4

STEP 4 OF 7 · CONT. 1/4

Save the refactored version

```
# sales_report.py
from typing import Iterable

Order = dict[str, object]

ORDERS: list[Order] = [
    {"id": 1, "region": "North", "amount": 1200.50, "status":
     "paid"},
    {"id": 2, "region": "South", "amount": 890.00, "status":
     "paid"},
    {"id": 3, "region": "North", "amount": 450.25, "status":
     "refunded"},
    {"id": 4, "region": "East", "amount": 2100.75, "status":
```

WHAT THIS DOES

- Read the signature first: what goes in, what comes out.
- The docstring states the contract the function promises.

Refactor Working Code Conversationally

4

STEP 4 OF 7 · CONT. 2/4

Save the refactored version

```
        "paid"},
        {"id": 5, "region": "South", "amount": 310.00, "status":
         "pending"},
    ]

    def filter_paid(orders: Iterable[Order]) -> list[Order]:
        """Return only the orders with status 'paid'."""
        return [o for o in orders if o["status"] == "paid"]

    def total_by_region(orders: Iterable[Order]) -> dict[str,
    float]:
        """Sum order amounts grouped by region."""
        totals: dict[str, float] = {}
```

WHAT THIS DOES

- Read the signature first: what goes in, what comes out.
- The docstring states the contract the function promises.

Refactor Working Code Conversationally

4

STEP 4 OF 7 · CONT. 3/4

Save the refactored version

```
for o in orders:
    region = str(o["region"])
    totals[region] = totals.get(region, 0.0) +
        float(o["amount"])
return totals

def format_report(totals: dict[str, float]) -> str:
    """Render the per-region totals and the grand total."""
    lines = [f"{r}: ${totals[r]:,.2f}" for r in
        sorted(totals)]
    lines.append(f"TOTAL: ${sum(totals.values()):,.2f}")
    return "\n".join(lines)
```

WHAT THIS DOES

- Read the signature first: what goes in, what comes out.
- The docstring states the contract the function promises.

Refactor Working Code Conversationally

4

STEP 4 OF 7 · CONT. 4/4

Save the refactored version

```
def main() -> None:
    print(format_report(total_by_region(filter_paid(ORDERS))))

if __name__ == "__main__":
    main()
```

WHAT THIS DOES

- Read the signature first: what goes in, what comes out.
- The docstring states the contract the function promises.

Refactor Working Code Conversationally

5

STEP 5 OF 7

Prove the behaviour is unchanged by diffing against the baseline

```
uv run python sales_report.py > after.txt  
diff baseline.txt after.txt && echo 'IDENTICAL – refactor is  
safe'
```

WHAT THIS DOES

- Run this in your terminal, from the project folder.
- `uv run` executes inside the project environment — no venv activation.

Refactor Working Code Conversationally

6

STEP 6 OF 7

Now that the logic is in functions, test one piece in isolation — impossible before the refactor

```
uv run python -c "  
from sales_report import total_by_region  
rows = [{'region': 'North', 'amount': 100.0, 'status':  
    'paid'},  
        {'region': 'North', 'amount': 50.0, 'status':  
    'paid'}]  
assert total_by_region(rows) == {'North': 150.0}  
print('unit test passed')  
"
```

WHAT THIS DOES

- Run this in your terminal, from the project folder.
- `uv run` executes inside the project environment — no `venv` activation.
- The assertion is the test — it must fail before you trust it.

Refactor Working Code Conversationally

7

STEP 7 OF 7

Note the sequence that makes refactoring safe: baseline first, refactor second, diff third.

Refactor Working Code Conversationally

Test it

diff reports no difference between baseline.txt and after.txt, and total_by_region can be unit-tested independently.

Recap — Vibe Coding Foundations

- You can now: Create and run a Python project using uv
- You can now: Apply prompting patterns that produce correct, reviewable code
- You can now: Identify and fix defects in code produced by an AI assistant
- You can now: Use an AI assistant to restructure code without changing its behaviour

TOPIC 02

Object-Oriented Programming in Python

Classes · Encapsulation · Inheritance · Composition · Dunder Methods

Key Concepts — Object-Oriented Programming in Python

1

A class bundles data (attributes) with the behaviour that operates on it (methods); an object is one instance of it.

2

Encapsulation hides internal state behind methods and properties so callers depend on behaviour, not layout.

3

Inheritance models 'is-a' and shares behaviour; composition models 'has-a' and is usually the more flexible default.

4

Dunder methods (`__init__`, `__repr__`, `__eq__`) integrate your classes with Python's built-in operators and printing.

Hands-On Labs — Object-Oriented Programming in Python

Labs 5–6

- Model a Domain with Classes and Dunder Methods
- Encapsulate State with Properties and Validation

Labs 7–8

- Build a Rule Hierarchy with Inheritance and Polymorphism
- Compose Rules into an Engine

—

- —

Model a Domain with Classes and Dunder Methods

LAB 5

The learner replaces loose dictionaries with a Transaction class, adds `__init__`, `__repr__` and `__eq__`, and sees why a class is easier to reason about than a dict once behaviour is attached to the data.

You'll build

A Transaction class with construction, printing, equality and derived properties

Tools & libraries: uv, Python 3.12, dataclasses, sqlite3

Model a Domain with Classes and Dunder Methods

1

STEP 1 OF 7

Start the project and generate the database

```
uv init cardguard  
cd cardguard  
uv add pandas  
cp ../mockdata.py .  
uv run python mockdata.py
```

WHAT THIS DOES

- Run this in your terminal, from the project folder.
- uv resolves, installs and records the dependency in uv.lock.
- uv run executes inside the project environment — no venv activation.

Model a Domain with Classes and Dunder Methods

2

STEP 2 OF 7

Look at the raw shape of the data you are about to model

```
uv run python -c "  
import sqlite3  
con = sqlite3.connect('cardguard.db')  
row = con.execute('SELECT * FROM transactions LIMIT  
1').fetchone()  
print(row)  
"
```

WHAT THIS DOES

- Run this in your terminal, from the project folder.
- `uv run` executes inside the project environment — no venv activation.

Model a Domain with Classes and Dunder Methods

3

STEP 3 OF 7 · CONT. 1/3

Write the Transaction class — data and the behaviour that belongs with it

```
# models.py
from dataclasses import dataclass
from datetime import datetime

@dataclass
class Transaction:
    """One card transaction presented for screening."""
    id: int
    card_ref: str
    ts: datetime
    amount: float
    merchant_category: str
    city: str
```

WHAT THIS DOES

- Read the signature first: what goes in, what comes out.
- The docstring states the contract the function promises.
- The class bundles data with the behaviour that acts on it.
- A property exposes state read-only — callers cannot assign to it.

Model a Domain with Classes and Dunder Methods

3

STEP 3 OF 7 · CONT. 2/3

Write the Transaction class — data and the behaviour that belongs with it

```
lat: float
lon: float

@property
def hour(self) -> int:
    """Hour of day, 0-23 – used by the unusual-hour
    rule."""
    return self.ts.hour

@property
def is_high_risk_category(self) -> bool:
    return self.merchant_category in {"crypto_exchange",
    "gift_cards", "jewellery"}
```

WHAT THIS DOES

- Read the signature first: what goes in, what comes out.
- The docstring states the contract the function promises.
- The class bundles data with the behaviour that acts on it.
- A property exposes state read-only — callers cannot assign to it.

Model a Domain with Classes and Dunder Methods

3

STEP 3 OF 7 · CONT. 3/3

Write the Transaction class — data and the behaviour that belongs with it

```
def __repr__(self) -> str:
    return (f"Transaction(id={self.id},
        card={self.card_ref}, "
        f"${self.amount:,.2f},
        {self.merchant_category}, {self.city})")
```

WHAT THIS DOES

- Read the signature first: what goes in, what comes out.
- The docstring states the contract the function promises.
- The class bundles data with the behaviour that acts on it.
- A property exposes state read-only — callers cannot assign to it.

Model a Domain with Classes and Dunder Methods

4

STEP 4 OF 7

Add a constructor that builds a Transaction from a database row

```
@classmethod
def from_row(cls, row: tuple) -> "Transaction":
    """Build a Transaction from a sqlite row tuple."""
    return cls(id=row[0], card_ref=row[1],
               ts=datetime.fromisoformat(row[2]),
               amount=row[3], merchant_category=row[4],
               city=row[5],
               lat=row[6], lon=row[7])
```

WHAT THIS DOES

- Read the signature first: what goes in, what comes out.
- The docstring states the contract the function promises.

Model a Domain with Classes and Dunder Methods

5

STEP 5 OF 7

Load real rows through the class and print them

```
uv run python -c "  
import sqlite3  
from models import Transaction  
con = sqlite3.connect('cardguard.db')  
rows = con.execute('SELECT * FROM transactions LIMIT  
5').fetchall()  
for r in rows:  
    t = Transaction.from_row(r)  
    print(t, '| hour', t.hour, '| high risk',  
          t.is_high_risk_category)  
"
```

WHAT THIS DOES

- Run this in your terminal, from the project folder.
- `uv run` executes inside the project environment — no `venv` activation.

Model a Domain with Classes and Dunder Methods

6

STEP 6 OF 7

Verify `@dataclass` gave you equality for free — dicts compare by content, objects normally by identity

```
uv run python -c "  
from datetime import datetime  
from models import Transaction  
a = Transaction(1, 'CH0001', datetime(2026,4,1,9), 50.0,  
    'grocery', 'Singapore', 1.35, 103.8)  
b = Transaction(1, 'CH0001', datetime(2026,4,1,9), 50.0,  
    'grocery', 'Singapore', 1.35, 103.8)  
print('equal:', a == b)  
print('same object:', a is b)  
"
```

WHAT THIS DOES

- Run this in your terminal, from the project folder.
- `uv run` executes inside the project environment — no `venv` activation.

Model a Domain with Classes and Dunder Methods

7

STEP 7 OF 7

Note the gain: `hour` and `is_high_risk_category` live WITH the data. With dicts, that logic would be scattered across every caller.

Model a Domain with Classes and Dunder Methods

Test it

Transaction.from_row builds objects from the database, __repr__ prints readably, __eq__ compares by value, and the derived properties return correct results.

Encapsulate State with Properties and Validation

LAB 6

The learner builds a Cardholder class whose running spend baseline cannot be corrupted from outside, using private attributes, a read-only property and validation in the setter.

You'll build

A Cardholder class with a protected baseline and a validated update method

Tools & libraries: uv, Python 3.12

Encapsulate State with Properties and Validation

1

STEP 1 OF 7 · CONT. 1/2

Write the class with state deliberately kept private

```
# cardholder.py
class Cardholder:
    """A card account with a running spend baseline used for
    anomaly scoring."""

    def __init__(self, card_ref: str, name: str, home_city:
        str, typical_amount: float):
        if typical_amount <= 0:
            raise ValueError("typical_amount must be
                positive")
        self.card_ref = card_ref
        self.name = name
        self.home_city = home_city
```

WHAT THIS DOES

- Read the signature first: what goes in, what comes out.
- The docstring states the contract the function promises.
- Invalid input fails fast and loudly, never silently.
- The class bundles data with the behaviour that acts on it.
- A property exposes state read-only — callers cannot assign to it.

Encapsulate State with Properties and Validation

1

STEP 1 OF 7 · CONT. 2/2

Write the class with state deliberately kept private

```
self._typical_amount = typical_amount    # leading _ =
    internal
self._observed_count = 0

@property
def typical_amount(self) -> float:
    """Read-only view of the baseline – callers cannot
       assign to it."""
    return round(self._typical_amount, 2)

@property
def observed_count(self) -> int:
    return self._observed_count
```

WHAT THIS DOES

- Read the signature first: what goes in, what comes out.
- The docstring states the contract the function promises.
- Invalid input fails fast and loudly, never silently.
- The class bundles data with the behaviour that acts on it.
- A property exposes state read-only — callers cannot assign to it.

Encapsulate State with Properties and Validation

2

STEP 2 OF 7 · CONT. 1/2

Add the only sanctioned way to move the baseline

```
def observe(self, amount: float) -> None:
    """Fold one new transaction into the running
    baseline."""
    if amount < 0:
        raise ValueError("amount must not be negative")
    self._observed_count += 1
    weight = 1 / min(self._observed_count, 30)    #
        rolling, recency-weighted
    self._typical_amount = (1 - weight) *
        self._typical_amount + weight * amount

def deviation_factor(self, amount: float) -> float:
    """How many times the baseline this amount
```

WHAT THIS DOES

- Read the signature first: what goes in, what comes out.
- The docstring states the contract the function promises.
- Invalid input fails fast and loudly, never silently.

Encapsulate State with Properties and Validation

2

STEP 2 OF 7 · CONT. 2/2

Add the only sanctioned way to move the baseline

```
represents."""  
return round(amount / self._typical_amount, 2)
```

WHAT THIS DOES

- Read the signature first: what goes in, what comes out.
- The docstring states the contract the function promises.
- Invalid input fails fast and loudly, never silently.

Encapsulate State with Properties and Validation

3

STEP 3 OF 7

Prove the baseline cannot be overwritten from outside

```
uv run python -c "  
from cardholder import Cardholder  
ch = Cardholder('CH0001', 'Aisha Tan', 'Singapore', 120.0)  
try:  
    ch.typical_amount = 999999  
except AttributeError as e:  
    print('blocked:', e)  
"
```

WHAT THIS DOES

- Run this in your terminal, from the project folder.
- `uv run` executes inside the project environment — no venv activation.
- `try/except` handles the failure; finally always cleans up.

Encapsulate State with Properties and Validation

4

STEP 4 OF 7

Drive the baseline the sanctioned way and watch it move

```
uv run python -c "  
from cardholder import Cardholder  
ch = Cardholder('CH0001', 'Aisha Tan', 'Singapore', 120.0)  
for amt in [110, 130, 125, 118, 122]:  
    ch.observe(amt)  
print('baseline', ch.typical_amount, 'after',  
      ch.observed_count, 'observations')  
print('a $2,400 charge is', ch.deviation_factor(2400), 'x  
baseline')  
"
```

WHAT THIS DOES

- Run this in your terminal, from the project folder.
- `uv run` executes inside the project environment — no `venv` activation.

Encapsulate State with Properties and Validation

5

STEP 5 OF 7

Confirm validation rejects bad input at both entry points

```
uv run python -c "  
from cardholder import Cardholder  
for bad in [lambda: Cardholder('C','n','SG',0), lambda:  
    Cardholder('C','n','SG',120).observe(-5)]:  
    try:  
        bad(); print('NOT CAUGHT')  
    except ValueError as e:  
        print('correctly raised:', e)  
"
```

WHAT THIS DOES

- Run this in your terminal, from the project folder.
- `uv run` executes inside the project environment — no venv activation.
- `try/except` handles the failure; finally always cleans up.

Encapsulate State with Properties and Validation

6

STEP 6 OF 7 · CONT. 1/2

Load the real cardholders and rank them by baseline

```
uv run python -c "  
import sqlite3  
from cardholder import Cardholder  
con = sqlite3.connect('cardguard.db')  
rows = con.execute('SELECT  
    card_ref,name,home_city,typical_amount FROM  
    cardholders').fetchall()  
chs = [Cardholder(*r) for r in rows]  
top = sorted(chs, key=lambda c: c.typical_amount,  
    reverse=True)[:5]  
for c in top:  
    print(f'{c.card_ref} {c.name:<16} baseline  
        ${c.typical_amount:,.2f}')
```

WHAT THIS DOES

- Run this in your terminal, from the project folder.
- `uv run` executes inside the project environment — no `venv` activation.

Encapsulate State with Properties and Validation

6

STEP 6 OF 7 · CONT. 2/2

Load the real cardholders and rank them by baseline

```
"
```

WHAT THIS DOES

- Run this in your terminal, from the project folder.
- `uv run` executes inside the project environment — no venv activation.

Encapsulate State with Properties and Validation

7

STEP 7 OF 7

Note why this matters for fraud: if any caller could assign `typical_amount` directly, an attacker-controlled input path could raise the baseline until nothing looks anomalous.

Encapsulate State with Properties and Validation

Test it

Assigning to `typical_amount` raises `AttributeError`; `observe()` updates the baseline; both constructors and `observe()` reject invalid values; the 5 highest baselines print from the database.

Build a Rule Hierarchy with Inheritance and Polymorphism

LAB 7

The learner defines an abstract FraudRule base class and implements four concrete rules. Each rule scores a transaction differently but presents an identical interface, so the caller never needs to know which rule it holds.

You'll build

An abstract FraudRule plus AmountDeviationRule, UnusualHourRule, HighRiskCategoryRule and VelocityRule

Tools & libraries: uv, Python 3.12, abc

Build a Rule Hierarchy with Inheritance and Polymorphism

1

STEP 1 OF 7 · CONT. 1/3

Define the contract every rule must satisfy

```
# rules.py
from abc import ABC, abstractmethod
from dataclasses import dataclass
from models import Transaction
from cardholder import Cardholder

@dataclass
class RuleResult:
    """One rule's verdict on one transaction."""
    rule_name: str
    score: float          # 0.0 (clean) .. 1.0 (certain)
    reason: str
```

WHAT THIS DOES

- Read the signature first: what goes in, what comes out.
- The docstring states the contract the function promises.
- The class bundles data with the behaviour that acts on it.
- A property exposes state read-only — callers cannot assign to it.
- Subclasses **MUST** implement this — the contract is enforced.

Build a Rule Hierarchy with Inheritance and Polymorphism

1

STEP 1 OF 7 · CONT. 2/3

Define the contract every rule must satisfy

```
class FraudRule(ABC):
    """Base class: every rule scores a transaction from 0.0
    to 1.0."""

    weight: float = 1.0

    @property
    def name(self) -> str:
        return self.__class__.__name__

    @abstractmethod
    def score(self, txn: Transaction, holder: Cardholder,
              history: list[Transaction]) -> RuleResult:
```

WHAT THIS DOES

- Read the signature first: what goes in, what comes out.
- The docstring states the contract the function promises.
- The class bundles data with the behaviour that acts on it.
- A property exposes state read-only — callers cannot assign to it.
- Subclasses **MUST** implement this — the contract is enforced.

Build a Rule Hierarchy with Inheritance and Polymorphism

1

STEP 1 OF 7 · CONT. 3/3

Define the contract every rule must satisfy

```
"""Return this rule's verdict. Must be implemented by  
every subclass."""
```

WHAT THIS DOES

- Read the signature first: what goes in, what comes out.
- The docstring states the contract the function promises.
- The class bundles data with the behaviour that acts on it.
- A property exposes state read-only — callers cannot assign to it.
- Subclasses **MUST** implement this — the contract is enforced.

Build a Rule Hierarchy with Inheritance and Polymorphism

2

STEP 2 OF 7

Prove the abstract class cannot be instantiated — the contract is enforced

```
uv run python -c "  
from rules import FraudRule  
try:  
    FraudRule()  
except TypeError as e:  
    print('correctly blocked:', e)  
"
```

WHAT THIS DOES

- Run this in your terminal, from the project folder.
- `uv run` executes inside the project environment — no `venv` activation.
- `try/except` handles the failure; finally always cleans up.

Build a Rule Hierarchy with Inheritance and Polymorphism

3

STEP 3 OF 7 · CONT. 1/2

Implement the amount-deviation rule

```
class AmountDeviationRule(FraudRule):
    """Flags spend far above this cardholder's own
    baseline."""
    weight = 1.2

    def __init__(self, alert_factor: float = 8.0):
        self.alert_factor = alert_factor

    def score(self, txn, holder, history):
        factor = holder.deviation_factor(txn.amount)
        s = min(factor / self.alert_factor, 1.0) if factor >
            1 else 0.0
        return RuleResult(self.name, round(s, 3),
```

WHAT THIS DOES

- Read the signature first: what goes in, what comes out.
- The docstring states the contract the function promises.
- The class bundles data with the behaviour that acts on it.

Build a Rule Hierarchy with Inheritance and Polymorphism

3

STEP 3 OF 7 · CONT. 2/2

Implement the amount-deviation rule

```
f"${txn.amount:,.2f} is {factor}x  
the ${holder.typical_amount:,.2f} baseline")
```

WHAT THIS DOES

- Read the signature first: what goes in, what comes out.
- The docstring states the contract the function promises.
- The class bundles data with the behaviour that acts on it.

Build a Rule Hierarchy with Inheritance and Polymorphism

4

STEP 4 OF 7 · CONT. 1/2

Implement the unusual-hour and high-risk-category rules

```
class UnusualHourRule(FraudRule):
    """Flags activity outside the cardholder's normal waking
    window."""
    weight = 0.6

    def score(self, txn, holder, history):
        s = 0.8 if txn.hour in {0, 1, 2, 3, 4, 5} else 0.0
        return RuleResult(self.name, s, f"transaction at
        {txn.hour:02d}:00")

class HighRiskCategoryRule(FraudRule):
    """Flags merchant categories with elevated chargeback
    rates."""
```

WHAT THIS DOES

- Read the signature first: what goes in, what comes out.
- The docstring states the contract the function promises.
- The class bundles data with the behaviour that acts on it.

Build a Rule Hierarchy with Inheritance and Polymorphism

4

STEP 4 OF 7 · CONT. 2/2

Implement the unusual-hour and high-risk-category rules

```
weight = 0.8

RISK = {"crypto_exchange": 0.9, "gift_cards": 0.8,
        "jewellery": 0.6, "online_gaming": 0.5}

def score(self, txn, holder, history):
    s = self.RISK.get(txn.merchant_category, 0.0)
    return RuleResult(self.name, s, f"category
        {txn.merchant_category}")
```

WHAT THIS DOES

- Read the signature first: what goes in, what comes out.
- The docstring states the contract the function promises.
- The class bundles data with the behaviour that acts on it.

Build a Rule Hierarchy with Inheritance and Polymorphism

5

STEP 5 OF 7 · CONT. 1/2

Implement the velocity rule — it needs history, which the shared interface already provides

```
from datetime import timedelta

class VelocityRule(FraudRule):
    """Flags many transactions inside a short window."""
    weight = 1.5

    def __init__(self, window_minutes: int = 10, threshold:
        int = 4):
        self.window = timedelta(minutes=window_minutes)
        self.threshold = threshold

    def score(self, txn, holder, history):
        recent = [h for h in history
```

WHAT THIS DOES

- Read the signature first: what goes in, what comes out.
- The docstring states the contract the function promises.
- The class bundles data with the behaviour that acts on it.

Build a Rule Hierarchy with Inheritance and Polymorphism

5

STEP 5 OF 7 · CONT. 2/2

Implement the velocity rule — it needs history, which the shared interface already provides

```
        if 0 <= (txn.ts - h.ts).total_seconds() <=
            self.window.total_seconds()]
    n = len(recent) + 1
    s = min((n - 1) / self.threshold, 1.0) if n > 1 else
        0.0
    return RuleResult(self.name, round(s, 3),
                      f"{n} transactions in
                      {self.window.seconds // 60} minutes")
```

WHAT THIS DOES

- Read the signature first: what goes in, what comes out.
- The docstring states the contract the function promises.
- The class bundles data with the behaviour that acts on it.

Build a Rule Hierarchy with Inheritance and Polymorphism

6

STEP 6 OF 7 · CONT. 1/2

Call every rule through the SAME interface — this is polymorphism doing real work

```
uv run python -c "  
import sqlite3  
from models import Transaction  
from cardholder import Cardholder  
from rules import AmountDeviationRule, UnusualHourRule,  
    HighRiskCategoryRule, VelocityRule  
con = sqlite3.connect('cardguard.db')  
row = con.execute('SELECT * FROM transactions WHERE  
    is_known_fraud=1 ORDER BY amount DESC LIMIT 1').fetchone()  
txn = Transaction.from_row(row)  
chr_ = con.execute('SELECT  
    card_ref,name,home_city,typical_amount FROM cardholders  
    WHERE card_ref=?', (txn.card_ref,)).fetchone()
```

WHAT THIS DOES

- Run this in your terminal, from the project folder.
- `uv run` executes inside the project environment — no venv activation.

Build a Rule Hierarchy with Inheritance and Polymorphism

6

STEP 6 OF 7 · CONT. 2/2

Call every rule through the SAME interface — this is polymorphism doing real work

```
holder = Cardholder(*chr_)
rules = [AmountDeviationRule(), UnusualHourRule(),
         HighRiskCategoryRule(), VelocityRule()]
print(txn)
for r in rules:
    res = r.score(txn, holder, [])
    print(f' {res.rule_name:<24} {res.score:>5}
          {res.reason}')
```

WHAT THIS DOES

- Run this in your terminal, from the project folder.
- `uv run` executes inside the project environment — no venv activation.

Build a Rule Hierarchy with Inheritance and Polymorphism

7

STEP 7 OF 7

Note what inheritance bought: adding a fifth rule requires no change to any caller.

Build a Rule Hierarchy with Inheritance and Polymorphism

Test it

FraudRule cannot be instantiated; all four rules implement score() and return a RuleResult; a known-fraud transaction produces non-zero scores from the relevant rules.

Compose Rules into an Engine

LAB 8

The learner builds a RuleEngine that HAS-A list of rules rather than inheriting from any of them, producing a weighted composite score with a full explanation of every contributing rule.

You'll build

A RuleEngine returning a ScreeningResult with a composite score and per-rule reasons

Tools & libraries: uv, Python 3.12

Compose Rules into an Engine

1

STEP 1 OF 7 · CONT. 1/2

Write the engine — note it inherits from nothing; it composes

```
# engine.py
from dataclasses import dataclass, field
from models import Transaction
from cardholder import Cardholder
from rules import FraudRule, RuleResult

@dataclass
class ScreeningResult:
    """The engine's decision, with every reason that produced
    it."""
    txn_id: int
    card_ref: str
    composite_score: float
```

WHAT THIS DOES

- Read the signature first: what goes in, what comes out.
- The docstring states the contract the function promises.
- The class bundles data with the behaviour that acts on it.

Compose Rules into an Engine

1

STEP 1 OF 7 · CONT. 2/2

Write the engine — note it inherits from nothing; it composes

```
flagged: bool
results: list[RuleResult] = field(default_factory=list)

def explain(self) -> str:
    head = (f"Transaction {self.txn_id}
           ({self.card_ref}): "
           f"score {self.composite_score} "
           f"{'FLAGGED' if self.flagged else 'clean'}")
    lines = [f"    - {r.rule_name} {r.score} ::
              {r.reason}"
             for r in self.results if r.score > 0]
    return "\n".join([head, *lines])
```

WHAT THIS DOES

- Read the signature first: what goes in, what comes out.
- The docstring states the contract the function promises.
- The class bundles data with the behaviour that acts on it.

Compose Rules into an Engine

2

STEP 2 OF 7 · CONT. 1/2

Add the engine itself

```
class RuleEngine:
    """Composes independent rules into one weighted
    decision."""

    def __init__(self, rules: list[FraudRule], threshold:
        float = 0.55):
        if not rules:
            raise ValueError("at least one rule is required")
        self.rules = rules
        self.threshold = threshold

    def screen(self, txn: Transaction, holder: Cardholder,
        history: list[Transaction]) -> ScreeningResult:
```

WHAT THIS DOES

- Read the signature first: what goes in, what comes out.
- The docstring states the contract the function promises.
- Invalid input fails fast and loudly, never silently.
- The class bundles data with the behaviour that acts on it.

Compose Rules into an Engine

2

STEP 2 OF 7 · CONT. 2/2

Add the engine itself

```
results = [r.score(txn, holder, history) for r in
            self.rules]
total_weight = sum(r.weight for r in self.rules)
composite = sum(res.score * rule.weight
                for res, rule in zip(results,
                                     self.rules)) / total_weight
composite = round(composite, 3)
return ScreeningResult(txn.id, txn.card_ref,
                       composite,
                       composite >= self.threshold,
                       results)
```

WHAT THIS DOES

- Read the signature first: what goes in, what comes out.
- The docstring states the contract the function promises.
- Invalid input fails fast and loudly, never silently.
- The class bundles data with the behaviour that acts on it.

Compose Rules into an Engine

3

STEP 3 OF 7 · CONT. 1/2

Screen a known-fraud transaction and read the explanation

```
uv run python -c "  
import sqlite3  
from models import Transaction  
from cardholder import Cardholder  
from rules import AmountDeviationRule, UnusualHourRule,  
    HighRiskCategoryRule, VelocityRule  
from engine import RuleEngine  
con = sqlite3.connect('cardguard.db')  
row = con.execute('SELECT * FROM transactions WHERE  
    is_known_fraud=1 ORDER BY amount DESC LIMIT 1').fetchone()  
txn = Transaction.from_row(row)  
holder = Cardholder(*con.execute('SELECT  
    card_ref,name,home_city,typical_amount FROM cardholders
```

WHAT THIS DOES

- Run this in your terminal, from the project folder.
- `uv run` executes inside the project environment — no venv activation.

Compose Rules into an Engine

3

STEP 3 OF 7 · CONT. 2/2

Screen a known-fraud transaction and read the explanation

```
WHERE card_ref=?',(txn.card_ref,)).fetchone())
eng = RuleEngine([AmountDeviationRule(), UnusualHourRule(),
    HighRiskCategoryRule(), VelocityRule()])
print(eng.screen(txn, holder, []).explain())
"
```

WHAT THIS DOES

- Run this in your terminal, from the project folder.
- `uv run` executes inside the project environment — no venv activation.

Compose Rules into an Engine

4

STEP 4 OF 7 · CONT. 1/2

Screen a normal transaction and confirm it stays clean

```
uv run python -c "  
import sqlite3  
from models import Transaction  
from cardholder import Cardholder  
from rules import AmountDeviationRule, UnusualHourRule,  
    HighRiskCategoryRule, VelocityRule  
from engine import RuleEngine  
con = sqlite3.connect('cardguard.db')  
row = con.execute("SELECT * FROM transactions WHERE  
    is_known_fraud=0 AND merchant_category='grocery' LIMIT  
    1").fetchone()  
txn = Transaction.from_row(row)  
holder = Cardholder(*con.execute('SELECT
```

WHAT THIS DOES

- Run this in your terminal, from the project folder.
- `uv run` executes inside the project environment — no venv activation.

Compose Rules into an Engine

4

STEP 4 OF 7 · CONT. 2/2

Screen a normal transaction and confirm it stays clean

```
card_ref,name,home_city,typical_amount FROM cardholders
WHERE card_ref=?',(txn.card_ref,)).fetchone()
eng = RuleEngine([AmountDeviationRule(), UnusualHourRule(),
HighRiskCategoryRule(), VelocityRule()])
print(eng.screen(txn, holder, []).explain())
"
```

WHAT THIS DOES

- Run this in your terminal, from the project folder.
- `uv run` executes inside the project environment — no venv activation.

Compose Rules into an Engine

5

STEP 5 OF 7

Reconfigure the engine WITHOUT touching any rule class — composition in action

```
uv run python -c "  
from rules import AmountDeviationRule, HighRiskCategoryRule  
from engine import RuleEngine  
strict = RuleEngine([AmountDeviationRule(alert_factor=4.0),  
    HighRiskCategoryRule()], threshold=0.35)  
print('rules:', [r.name for r in strict.rules], 'threshold:',  
    strict.threshold)  
"
```

WHAT THIS DOES

- Run this in your terminal, from the project folder.
- `uv run` executes inside the project environment — no venv activation.

Compose Rules into an Engine

6

STEP 6 OF 7

Ask your AI assistant to add a fifth rule and confirm the engine needs no edit

```
Ask the assistant: Add a RoundAmountRule to rules.py that
subclasses FraudRule with weight 0.4 and returns score
0.5 when the transaction amount is an exact multiple of
100, since round-number testing charges are a common
card-testing signal. Do not modify RuleEngine.
```

WHAT THIS DOES

- Ask your AI assistant to add a fifth rule and confirm the engine needs no edit

Compose Rules into an Engine

7

STEP 7 OF 7

Note the design principle: inheritance shares WHAT rules are; composition decides WHICH rules run.

Compose Rules into an Engine

Test it

The engine flags the known-fraud transaction with a readable explanation, leaves a grocery transaction clean, and can be reconfigured with different rules and thresholds without editing any rule class.

Recap — Object-Oriented Programming in Python

- You can now: Define classes that bundle data with behaviour
- You can now: Protect object state behind a controlled interface
- You can now: Share behaviour through a base class and override it per rule
- You can now: Use composition to assemble behaviour from independent parts

TOPIC 03

Data Analytics with pandas

DataFrames · Cleaning · Transformation · GroupBy · Aggregation · Reporting

Key Concepts — Data Analytics with pandas

1

A DataFrame is a labelled 2-D table; a Series is one column. Most analytics is selecting, filtering and reshaping these.

2

Real data is dirty: missing values, wrong dtypes and duplicates must be handled explicitly before analysis.

3

split-apply-combine (groupby then aggregate) answers most business questions about a dataset.

4

An analytics pipeline should be a set of small, testable functions — not one long script.

Hands-On Labs — Data Analytics with pandas

Labs 9–10

- Load and Explore Transactions with pandas
- Handle Errors with Exceptions

Labs 11–12

- Build Per-Cardholder Baselines with GroupBy
- Detect Velocity Bursts with Rolling Time Windows

—

- —

Load and Explore Transactions with pandas

LAB 9

The learner loads the transaction table into pandas, inspects dtypes and null counts, and produces a first profile of spending — the step every analytics task starts with.

You'll build

A loaded, correctly-typed transactions DataFrame plus a spend profile by category

Tools & libraries: uv, pandas 3.0, sqlite3

Load and Explore Transactions with pandas

1

STEP 1 OF 7

Add pandas and load the table

```
uv add pandas
uv run python -c "
import sqlite3, pandas as pd
con = sqlite3.connect('cardguard.db')
df = pd.read_sql_query('SELECT * FROM transactions', con)
print(df.shape)
print(df.head())
"
```

WHAT THIS DOES

- Run this in your terminal, from the project folder.
- uv resolves, installs and records the dependency in uv.lock.
- uv run executes inside the project environment — no venv activation.

Load and Explore Transactions with pandas

2

STEP 2 OF 7

Inspect dtypes — note ts arrived as text, not datetime

```
uv run python -c "  
import sqlite3, pandas as pd  
con = sqlite3.connect('cardguard.db')  
df = pd.read_sql_query('SELECT * FROM transactions', con)  
print(df.dtypes)  
print(df.isna().sum())  
"
```

WHAT THIS DOES

- Run this in your terminal, from the project folder.
- `uv run` executes inside the project environment — no venv activation.

Load and Explore Transactions with pandas

3

STEP 3 OF 7 · CONT. 1/2

Write a loader that fixes the types once, so no downstream code has to

```
# analytics.py
import sqlite3
import pandas as pd

def load_transactions(db_path: str = "cardguard.db") ->
    pd.DataFrame:
    """Load transactions with correct dtypes."""
    con = sqlite3.connect(db_path)
    try:
        df = pd.read_sql_query("SELECT * FROM transactions",
                               con)
    finally:
        con.close()           # always closes, even if the
```

WHAT THIS DOES

- Read the signature first: what goes in, what comes out.
- The docstring states the contract the function promises.
- try/except handles the failure; finally always cleans up.

Load and Explore Transactions with pandas

3

STEP 3 OF 7 · CONT. 2/2

Write a loader that fixes the types once, so no downstream code has to

```
        query raises
df["ts"] = pd.to_datetime(df["ts"])
df["merchant_category"] =
    df["merchant_category"].astype("category")
df["hour"] = df["ts"].dt.hour
df["date"] = df["ts"].dt.date
return df
```

WHAT THIS DOES

- Read the signature first: what goes in, what comes out.
- The docstring states the contract the function promises.
- try/except handles the failure; finally always cleans up.

Load and Explore Transactions with pandas

4

STEP 4 OF 7

Profile spend by merchant category — split-apply-combine

```
uv run python -c "  
from analytics import load_transactions  
df = load_transactions()  
profile = (df.groupby('merchant_category',  
    observed=True)['amount']  
           .agg(['count', 'sum', 'mean', 'max']).round(2)  
           .sort_values('sum', ascending=False))  
print(profile)  
"
```

WHAT THIS DOES

- Run this in your terminal, from the project folder.
- `uv run` executes inside the project environment — no venv activation.
- split-apply-combine: group the rows, then aggregate each group.

Load and Explore Transactions with pandas

5

STEP 5 OF 7

Answer a real business question: which hours carry the most spend?

```
uv run python -c "  
from analytics import load_transactions  
df = load_transactions()  
by_hour =  
    df.groupby('hour')['amount'].agg(['count', 'sum']).round(2)  
print(by_hour.tail(8))  
print('\novernight share:',  
      round(100*df[df.hour<6]['amount'].sum()/df['amount'].sum()  
            ,2), '%')  
"
```

WHAT THIS DOES

- Run this in your terminal, from the project folder.
- `uv run` executes inside the project environment — no venv activation.
- split-apply-combine: group the rows, then aggregate each group.

Load and Explore Transactions with pandas

6

STEP 6 OF 7

See the pandas 3.0 Copy-on-Write behaviour that AI assistants get wrong

```
uv run python -c "  
import pandas as pd  
df = pd.DataFrame({'a':[1,2,3], 'b':[10,20,30]})  
# The pandas 1.x idiom an assistant will often generate:  
df[df.a > 1]['b'] = 0  
print(df)    # unchanged under Copy-on-Write – the write went  
              to a temporary  
# The correct form:  
df.loc[df.a > 1, 'b'] = 0  
print(df)  
"
```

WHAT THIS DOES

- Run this in your terminal, from the project folder.
- `uv run` executes inside the project environment — no `venv` activation.

Load and Explore Transactions with pandas

7

STEP 7 OF 7

Note the lesson: verify generated pandas against your installed version, not against a tutorial.

Load and Explore Transactions with pandas

Test it

`load_transactions` returns 10,851 rows with `ts` as `datetime64`; the category profile and hourly breakdown print; the learner can state why chained assignment silently fails.

Handle Errors with Exceptions

LAB 10

The learner makes the loader robust: catching the specific failures that actually occur, defining a custom exception hierarchy for the domain, and guaranteeing cleanup with finally. A screening service that crashes on one bad row fails open — this lab prevents that.

You'll build

A domain exception hierarchy plus a validated, fail-safe loader

Tools & libraries: uv, pandas, sqlite3, Python exceptions

Handle Errors with Exceptions

1

STEP 1 OF 7 · CONT. 1/2

Define exceptions that describe the DOMAIN, not the plumbing

```
# errors.py
class CardGuardError(Exception):
    """Base class for every CardGuard failure – callers can
    catch this one type."""

    class DataSourceError(CardGuardError):
        """The transaction store could not be read."""

    class ValidationError(CardGuardError):
        """A transaction failed validation before scoring."""
        def __init__(self, field: str, value, message: str):
            self.field, self.value = field, value
            super().__init__(f"{field}={value!r}: {message}")
```

WHAT THIS DOES

- Read the signature first: what goes in, what comes out.
- The docstring states the contract the function promises.
- The class bundles data with the behaviour that acts on it.

Handle Errors with Exceptions

1

STEP 1 OF 7 · CONT. 2/2

Define exceptions that describe the DOMAIN, not the plumbing

```
class RuleExecutionError(CardGuardError):  
    """A scoring rule raised while screening a transaction."""
```

WHAT THIS DOES

- Read the signature first: what goes in, what comes out.
- The docstring states the contract the function promises.
- The class bundles data with the behaviour that acts on it.

Handle Errors with Exceptions

2

STEP 2 OF 7 · CONT. 1/3

Catch the SPECIFIC errors that happen, and translate them to domain errors

```
# analytics.py (updated)
import sqlite3
from pathlib import Path
import pandas as pd
from errors import DataSourceError

def load_transactions(db_path: str = "cardguard.db") ->
    pd.DataFrame:
    """Load transactions, raising DataSourceError on any read
    failure."""
    if not Path(db_path).exists():
        raise DataSourceError(f"database not found:
        {db_path}")
```

WHAT THIS DOES

- Read the signature first: what goes in, what comes out.
- The docstring states the contract the function promises.
- Invalid input fails fast and loudly, never silently.
- try/except handles the failure; finally always cleans up.

Handle Errors with Exceptions

2

STEP 2 OF 7 · CONT. 2/3

Catch the SPECIFIC errors that happen, and translate them to domain errors

```
con = None
try:
    con = sqlite3.connect(db_path)
    df = pd.read_sql_query("SELECT * FROM transactions",
                           con)
except sqlite3.DatabaseError as exc:
    raise DataSourceError(f"could not read {db_path}:
                          {exc}") from exc
else:
    df["ts"] = pd.to_datetime(df["ts"])
    df["hour"] = df["ts"].dt.hour
    return df
finally:
```

WHAT THIS DOES

- Read the signature first: what goes in, what comes out.
- The docstring states the contract the function promises.
- Invalid input fails fast and loudly, never silently.
- try/except handles the failure; finally always cleans up.

Handle Errors with Exceptions

2

STEP 2 OF 7 · CONT. 3/3

Catch the SPECIFIC errors that happen, and translate them to domain errors

```
if con is not None:  
    con.close()      # runs on success AND on failure
```

WHAT THIS DOES

- Read the signature first: what goes in, what comes out.
- The docstring states the contract the function promises.
- Invalid input fails fast and loudly, never silently.
- try/except handles the failure; finally always cleans up.

Handle Errors with Exceptions

3

STEP 3 OF 7

Prove each failure path raises the right domain error

```
uv run python -c "  
from analytics import load_transactions  
from errors import DataSourceError  
try:  
    load_transactions('no_such.db')  
except DataSourceError as e:  
    print('missing file ->', e)  
open('corrupt.db', 'wb').write(b'not a database at all')  
try:  
    load_transactions('corrupt.db')  
except DataSourceError as e:  
    print('corrupt file ->', type(e).__name__)  
"
```

WHAT THIS DOES

- Run this in your terminal, from the project folder.
- `uv run` executes inside the project environment — no venv activation.
- `try/except` handles the failure; finally always cleans up.

Handle Errors with Exceptions

4

STEP 4 OF 7 · CONT. 1/3

Validate a transaction and raise `ValidationError` carrying the offending field

```
# validate.py
from errors import ValidationError

VALID_CATEGORIES =
    {"grocery", "fuel", "restaurant", "electronics", "online_gaming",
     "jewellery", "crypto_exchange", "gift_cards",
     "pharmacy", "transport"}

def validate_txn(record: dict) -> dict:
    """Raise ValidationError on the first problem found."""
    amount = record.get("amount")
```

WHAT THIS DOES

- Read the signature first: what goes in, what comes out.
- The docstring states the contract the function promises.
- Invalid input fails fast and loudly, never silently.

Handle Errors with Exceptions

4

STEP 4 OF 7 · CONT. 2/3

Validate a transaction and raise `ValidationError` carrying the offending field

```
if amount is None:
    raise ValidationError("amount", amount, "is required")
if not isinstance(amount, (int, float)):
    raise ValidationError("amount", amount, "must be
    numeric")
if amount <= 0:
    raise ValidationError("amount", amount, "must be
    positive")
if amount > 1_000_000:
    raise ValidationError("amount", amount, "exceeds the
    maximum single charge")
cat = record.get("merchant_category")
if cat not in VALID_CATEGORIES:
```

WHAT THIS DOES

- Read the signature first: what goes in, what comes out.
- The docstring states the contract the function promises.
- Invalid input fails fast and loudly, never silently.

Handle Errors with Exceptions

4

STEP 4 OF 7 · CONT. 3/3

Validate a transaction and raise `ValidationError` carrying the offending field

```
raise ValidationError("merchant_category", cat, "is  
    not a known category")  
return record
```

WHAT THIS DOES

- Read the signature first: what goes in, what comes out.
- The docstring states the contract the function promises.
- Invalid input fails fast and loudly, never silently.

Handle Errors with Exceptions

5

STEP 5 OF 7 · CONT. 1/2

Run a batch where some records are bad — the batch must NOT die on the first failure

```
uv run python -c "  
from validate import validate_txn  
from errors import ValidationError  
batch = [  
    {'amount': 120.0, 'merchant_category': 'grocery'},  
    {'amount': -5.0, 'merchant_category': 'fuel'},  
    {'amount': 'abc', 'merchant_category': 'grocery'},  
    {'amount': 90.0, 'merchant_category': 'casino'},  
    {'amount': 45.0, 'merchant_category': 'transport'},  
]  
ok, rejected = [], []  
for i, rec in enumerate(batch):  
    try:
```

WHAT THIS DOES

- Run this in your terminal, from the project folder.
- `uv run` executes inside the project environment — no venv activation.
- `try/except` handles the failure; finally always cleans up.

Handle Errors with Exceptions

5

STEP 5 OF 7 · CONT. 2/2

Run a batch where some records are bad — the batch must NOT die on the first failure

```
        ok.append(validate_txn(rec))
    except ValidationError as e:
        rejected.append((i, str(e)))
print(f'accepted {len(ok)}, rejected {len(rejected)}')
for i, msg in rejected:
    print(f'  row {i}: {msg}')
```

WHAT THIS DOES

- Run this in your terminal, from the project folder.
- `uv run` executes inside the project environment — no venv activation.
- `try/except` handles the failure; finally always cleans up.

Handle Errors with Exceptions

6

STEP 6 OF 7 · CONT. 1/2

Contrast with the anti-pattern — a bare except hides the bug you needed to see

```
uv run python -c "  
# ANTI-PATTERN: never do this  
try:  
    result = 1 / 0  
except:          # catches everything, including typos  
    and KeyboardInterrupt  
    result = 0  
print('silently wrong:', result)  
  
# Correct: name the exception you expect and can handle  
try:  
    result = 1 / 0  
except ZeroDivisionError as e:
```

WHAT THIS DOES

- Run this in your terminal, from the project folder.
- `uv run` executes inside the project environment — no venv activation.
- `try/except` handles the failure; finally always cleans up.

Handle Errors with Exceptions

6

STEP 6 OF 7 · CONT. 2/2

Contrast with the anti-pattern — a bare except hides the bug you needed to see

```
" print('handled explicitly:', e)
```

WHAT THIS DOES

- Run this in your terminal, from the project folder.
- `uv run` executes inside the project environment — no venv activation.
- `try/except` handles the failure; finally always cleans up.

Handle Errors with Exceptions

7

STEP 7 OF 7

Ask your assistant to add a rule-level guard so one broken rule cannot stop a screen

Ask the assistant: In `RuleEngine.screen`, wrap each rule call in `try/except` so that if one rule raises, the engine records a `RuleExecutionError` message on the result and continues scoring with the remaining rules. Never use a bare `except`.

WHAT THIS DOES

- Ask your assistant to add a rule-level guard so one broken rule cannot stop a screen

Handle Errors with Exceptions

Test it

Missing and corrupt databases raise `DataSourceError`; the 5-record batch accepts 2 and rejects 3 with field-level messages; the learner can explain why bare except is unsafe.

Build Per-Cardholder Baselines with GroupBy

LAB 11

The learner computes each cardholder's own spending baseline and deviation for every transaction — the statistical foundation the `AmountDeviationRule` needs, done for 40 cardholders at once instead of one at a time.

You'll build

A per-cardholder baseline table and a deviation column on every transaction

Tools & libraries: uv, pandas 3.0

Build Per-Cardholder Baselines with GroupBy

1

STEP 1 OF 6

Compute one baseline per cardholder

```
uv run python -c "  
from analytics import load_transactions  
df = load_transactions()  
base =  
    df.groupby('card_ref')['amount'].agg(['count', 'mean', 'medi  
    an', 'std']).round(2)  
print(base.head())  
print('cardholders:', len(base))  
"
```

WHAT THIS DOES

- Run this in your terminal, from the project folder.
- `uv run` executes inside the project environment — no venv activation.
- split-apply-combine: group the rows, then aggregate each group.

Build Per-Cardholder Baselines with GroupBy

2

STEP 2 OF 6

Join each transaction to its cardholder baseline with transform — same shape as the original

```
# analytics.py (add)
def add_deviation(df):
    """Attach each transaction's deviation from its own
    cardholder baseline."""
    df = df.copy()
    df["baseline"] =
        df.groupby("card_ref")["amount"].transform("median")
    df["dev_factor"] = (df["amount"] /
        df["baseline"]).round(2)
    return df
```

WHAT THIS DOES

- Read the signature first: what goes in, what comes out.
- The docstring states the contract the function promises.
- split-apply-combine: group the rows, then aggregate each group.

Build Per-Cardholder Baselines with GroupBy

3

STEP 3 OF 6

Find the biggest deviations and check them against the seeded fraud flag

```
uv run python -c "  
from analytics import load_transactions, add_deviation  
df = add_deviation(load_transactions())  
top = df.nlargest(10,  
    'dev_factor')[['id', 'card_ref', 'amount', 'baseline', 'dev_fa  
    ctor', 'merchant_category', 'is_known_fraud']]  
print(top.to_string(index=False))  
print('\nof the top 10 deviations,',  
    int(top.is_known_fraud.sum()), 'are seeded fraud')  
"
```

WHAT THIS DOES

- Run this in your terminal, from the project folder.
- `uv run` executes inside the project environment — no `venv` activation.

Build Per-Cardholder Baselines with GroupBy

4

STEP 4 OF 6

Measure how well deviation alone separates fraud from normal

```
uv run python -c "  
from analytics import load_transactions, add_deviation  
df = add_deviation(load_transactions())  
print(df.groupby('is_known_fraud')['dev_factor'].describe().ro  
und(2)[['count', 'mean', '50%', 'max']])  
"
```

WHAT THIS DOES

- Run this in your terminal, from the project folder.
- `uv run` executes inside the project environment — no venv activation.
- split-apply-combine: group the rows, then aggregate each group.

Build Per-Cardholder Baselines with GroupBy

5

STEP 5 OF 6

Use median not mean for the baseline — prove why with an outlier

```
uv run python -c "  
import pandas as pd  
s = pd.Series([100,110,95,105,20000])  
print('mean ', round(s.mean(),2), '<- dragged up by the  
  fraud itself')  
print('median', round(s.median(),2), '<- robust to the  
  outlier')  
"
```

WHAT THIS DOES

- Run this in your terminal, from the project folder.
- `uv run` executes inside the project environment — no `venv` activation.

Build Per-Cardholder Baselines with GroupBy

6

STEP 6 OF 6

Note the analytics lesson: the statistic you choose changes what you can detect.

Build Per-Cardholder Baselines with GroupBy

Test it

Baselines compute for all 40 cardholders; dev_factor is attached to every row; the top deviations are dominated by seeded fraud; the learner can justify median over mean.

Detect Velocity Bursts with Rolling Time Windows

LAB 12

The learner sorts transactions per cardholder and counts how many occur inside a rolling 10-minute window — the history the VelocityRule needs, which cannot be computed one transaction at a time.

You'll build

A `txn_count_10min` column plus the burst transactions it exposes

Tools & libraries: uv, pandas 3.0, rolling windows

Detect Velocity Bursts with Rolling Time Windows

1

STEP 1 OF 7

Sort by cardholder and time — order is a precondition for any window function

```
# analytics.py (add)
def add_velocity(df, window: str = "10min"):
    """Count transactions per cardholder inside a rolling
       time window."""
    df = df.sort_values(["card_ref", "ts"]).copy()
    counts = (df.set_index("ts")
              .groupby("card_ref")["amount"]
              .rolling(window).count()
              .reset_index(name="txn_count_10min"))
    df = df.merge(counts, on=["card_ref", "ts"], how="left")
    return df
```

WHAT THIS DOES

- Read the signature first: what goes in, what comes out.
- The docstring states the contract the function promises.
- split-apply-combine: group the rows, then aggregate each group.

Detect Velocity Bursts with Rolling Time Windows

2

STEP 2 OF 7

Find the bursts

```
uv run python -c "  
from analytics import load_transactions, add_velocity  
df = add_velocity(load_transactions())  
bursts = df[df.txn_count_10min >= 4]  
print('burst transactions:', len(bursts))  
print(bursts[['card_ref', 'ts', 'amount', 'merchant_category', 'tx  
n_count_10min', 'is_known_fraud']].head(12).to_string(index  
=False))  
"
```

WHAT THIS DOES

- Run this in your terminal, from the project folder.
- `uv run` executes inside the project environment — no venv activation.

Detect Velocity Bursts with Rolling Time Windows

3

STEP 3 OF 7

Check the hit rate of velocity as a signal on its own

```
uv run python -c "  
from analytics import load_transactions, add_velocity  
df = add_velocity(load_transactions())  
b = df[df.txn_count_10min >= 4]  
print(f'{int(b.is_known_fraud.sum())} of {len(b)} burst  
transactions are seeded fraud')  
print('precision:', round(100*b.is_known_fraud.mean(),1), '%')  
"
```

WHAT THIS DOES

- Run this in your terminal, from the project folder.
- `uv run` executes inside the project environment — no venv activation.

Detect Velocity Bursts with Rolling Time Windows

4

STEP 4 OF 7

Inspect one full burst to see the pattern a fraudster leaves

```
uv run python -c "  
from analytics import load_transactions, add_velocity  
df = add_velocity(load_transactions())  
card = df[df.txn_count_10min >= 5].card_ref.iloc[0]  
win = df[df.card_ref == card].nlargest(8,  
    'txn_count_10min')[['ts', 'amount', 'merchant_category', 'txn  
    _count_10min']]  
print(f'card {card}');  
    print(win.sort_values('ts').to_string(index=False))  
"
```

WHAT THIS DOES

- Run this in your terminal, from the project folder.
- `uv run` executes inside the project environment — no `venv` activation.

Detect Velocity Bursts with Rolling Time Windows

5

STEP 5 OF 7 · CONT. 1/2

Compute the geographic-impossibility signal the engine is still missing

```
# analytics.py (add)
import numpy as np

def add_geo_velocity(df):
    """Implied travel speed (km/h) between consecutive
    transactions."""
    df = df.sort_values(["card_ref", "ts"]).copy()
    g = df.groupby("card_ref")
    lat1, lon1 = np.radians(g["lat"].shift()),
        np.radians(g["lon"].shift())
    lat2, lon2 = np.radians(df["lat"]), np.radians(df["lon"])
    dlat, dlon = lat2 - lat1, lon2 - lon1
    a = np.sin(dlat/2)**2 +
```

WHAT THIS DOES

- Read the signature first: what goes in, what comes out.
- The docstring states the contract the function promises.
- split-apply-combine: group the rows, then aggregate each group.

Detect Velocity Bursts with Rolling Time Windows

5

STEP 5 OF 7 · CONT. 2/2

Compute the geographic-impossibility signal the engine is still missing

```
np.cos(lat1)*np.cos(lat2)*np.sin(dlon/2)**2
km = 6371 * 2 * np.arcsin(np.sqrt(a))
hours = g["ts"].diff().dt.total_seconds() / 3600
df["implied_kmh"] = (km / hours.replace(0,
    np.nan)).round(1)
return df
```

WHAT THIS DOES

- Read the signature first: what goes in, what comes out.
- The docstring states the contract the function promises.
- split-apply-combine: group the rows, then aggregate each group.

Detect Velocity Bursts with Rolling Time Windows

6

STEP 6 OF 7

Find the physically impossible journeys

```
uv run python -c "  
from analytics import load_transactions, add_geo_velocity  
df = add_geo_velocity(load_transactions())  
imp = df[df.implicit_kmh > 1000]  
print('impossible journeys:', len(imp))  
print(imp[['card_ref', 'ts', 'city', 'amount', 'implicit_kmh', 'is_k  
nown_fraud']].to_string(index=False))  
"
```

WHAT THIS DOES

- Run this in your terminal, from the project folder.
- `uv run` executes inside the project environment — no venv activation.

Detect Velocity Bursts with Rolling Time Windows

7

STEP 7 OF 7

Note: velocity and geography are only visible ACROSS rows. Row-at-a-time scoring cannot see them.

Detect Velocity Bursts with Rolling Time Windows

Test it

Velocity bursts and impossible journeys are both detected, dominated by seeded fraud, and the learner can explain why these signals require sorted per-group windows.

Recap — Data Analytics with pandas

- You can now: Read SQL into a DataFrame and profile it
- You can now: Use try/except/else/finally and raise meaningful custom exceptions
- You can now: Compute per-group statistics with split-apply-combine
- You can now: Analyse ordered time-series data per group

TOPIC 04

Data Modelling with Pydantic and FastAPI

Typed Models · Validation · Endpoints · Request/Response Schemas · Auto Docs

Key Concepts — Data Modelling with Pydantic and FastAPI

1

Pydantic models declare the shape of data with Python type hints and validate it at runtime, failing fast on bad input.

2

FastAPI turns typed Python functions into HTTP endpoints, deriving validation and OpenAPI docs from the annotations.

3

Separate request models from response models so the API contract is explicit and safe to change.

4

The analytics layer stays plain Python; FastAPI is a thin transport layer over it.

Hands-On Labs — Data Modelling with Pydantic and FastAPI

Labs 13–14

- Replace Hand-Written Validation with Pydantic Models
- Expose the Screening Engine as a FastAPI Endpoint

Labs 15–16

- Persist Screening Decisions to SQLite
- Test the API with pytest and httpx

—

- —

Replace Hand-Written Validation with Pydantic Models

LAB 13

The learner replaces the manual `validate_txn()` from Topic 3 with a Pydantic model, getting type coercion, field constraints and structured error reporting for free — far less code, far better errors.

You'll build

TransactionIn and ScreeningOut Pydantic models with field constraints

Tools & libraries: uv, Pydantic v2

Replace Hand-Written Validation with Pydantic Models

1

STEP 1 OF 8

Add Pydantic to the project

```
uv add pydantic
```

WHAT THIS DOES

- Run this in your terminal, from the project folder.
- uv resolves, installs and records the dependency in uv.lock.

Replace Hand-Written Validation with Pydantic Models

2

STEP 2 OF 8 · CONT. 1/3

Declare the model — the constraints ARE the validation

```
# schemas.py
from datetime import datetime
from typing import Literal
from pydantic import BaseModel, Field, field_validator

CATEGORIES =
    Literal["grocery", "fuel", "restaurant", "electronics", "online_gaming",
            "jewellery", "crypto_exchange", "gift_cards",
            "pharmacy", "transport"]

class TransactionIn(BaseModel):
```

WHAT THIS DOES

- Read the signature first: what goes in, what comes out.
- The docstring states the contract the function promises.
- The class bundles data with the behaviour that acts on it.
- Pydantic validates the data at runtime from the type hints.

Replace Hand-Written Validation with Pydantic Models

2

STEP 2 OF 8 · CONT. 2/3

Declare the model — the constraints ARE the validation

```
"""One transaction submitted for screening."""
card_ref: str = Field(pattern=r"^CH\d{4}$",
    description="Cardholder reference")
ts: datetime
amount: float = Field(gt=0, le=1_000_000,
    description="Charge amount in SGD")
merchant_category: CATEGORIES
city: str = Field(min_length=2, max_length=64)
lat: float = Field(ge=-90, le=90)
lon: float = Field(ge=-180, le=180)

@field_validator("city")
@classmethod
```

WHAT THIS DOES

- Read the signature first: what goes in, what comes out.
- The docstring states the contract the function promises.
- The class bundles data with the behaviour that acts on it.
- Pydantic validates the data at runtime from the type hints.

Replace Hand-Written Validation with Pydantic Models

2

STEP 2 OF 8 · CONT. 3/3

Declare the model — the constraints ARE the validation

```
def title_case_city(cls, v: str) -> str:  
    return v.strip().title()
```

WHAT THIS DOES

- Read the signature first: what goes in, what comes out.
- The docstring states the contract the function promises.
- The class bundles data with the behaviour that acts on it.
- Pydantic validates the data at runtime from the type hints.

Replace Hand-Written Validation with Pydantic Models

3

STEP 3 OF 8

Compare against Topic 3 — one model replaces the whole hand-written validator

```
uv run python -c "  
from schemas import TransactionIn  
good = TransactionIn(card_ref='CH0001',  
    ts='2026-04-15T02:14:00', amount=4820.50,  
    merchant_category='jewellery',  
    city='singapore', lat=1.3521, lon=103.8198)  
print(good)  
print('city normalised to:', good.city)  
print('ts coerced to:', type(good.ts).__name__)  
"
```

WHAT THIS DOES

- Run this in your terminal, from the project folder.
- `uv run` executes inside the project environment — no `venv` activation.

Replace Hand-Written Validation with Pydantic Models

4

STEP 4 OF 8

Read a validation failure — Pydantic reports EVERY problem, not just the first

```
uv run python -c "  
from pydantic import ValidationError  
from schemas import TransactionIn  
try:  
    TransactionIn(card_ref='BAD', ts='not-a-date', amount=-5,  
                  merchant_category='casino', city='X',  
                  lat=999, lon=0)  
except ValidationError as e:  
    for err in e.errors():  
        print(f\" {'.'.join(str(x) for x in err['loc']):<20}  
              {err['msg']}\")  
"
```

WHAT THIS DOES

- Run this in your terminal, from the project folder.
- `uv run` executes inside the project environment — no venv activation.
- `try/except` handles the failure; finally always cleans up.

Replace Hand-Written Validation with Pydantic Models

5

STEP 5 OF 8 · CONT. 1/2

Define the response model separately — request and response are different contracts

```
# schemas.py (add)
class RuleHit(BaseModel):
    rule_name: str
    score: float = Field(ge=0, le=1)
    reason: str

class ScreeningOut(BaseModel):
    """What the screening service returns."""
    card_ref: str
    composite_score: float = Field(ge=0, le=1)
    flagged: bool
    decision: Literal["approve", "review", "decline"]
    hits: list[RuleHit] = []
```

WHAT THIS DOES

- The docstring states the contract the function promises.
- The class bundles data with the behaviour that acts on it.
- Pydantic validates the data at runtime from the type hints.

Replace Hand-Written Validation with Pydantic Models

5

STEP 5 OF 8 · CONT. 2/2

Define the response model separately — request and response are different contracts

```
errors: list[str] = []
```

WHAT THIS DOES

- The docstring states the contract the function promises.
- The class bundles data with the behaviour that acts on it.
- Pydantic validates the data at runtime from the type hints.

Replace Hand-Written Validation with Pydantic Models

6

STEP 6 OF 8

Serialise to JSON — the wire format comes free

```
uv run python -c "  
from schemas import ScreeningOut, RuleHit  
out = ScreeningOut(card_ref='CH0023', composite_score=0.933,  
    flagged=True, decision='decline',  
  
    hits=[RuleHit(rule_name='AmountDeviationRu  
le', score=1.0, reason='26.89x baseline'),  
          RuleHit(rule_name='UnusualHourRule',  
score=0.8, reason='02:00')])  
print(out.model_dump_json(indent=2))  
"
```

WHAT THIS DOES

- Run this in your terminal, from the project folder.
- `uv run` executes inside the project environment — no venv activation.

Replace Hand-Written Validation with Pydantic Models

7

STEP 7 OF 8 · CONT. 1/2

Load real database rows THROUGH the model to catch any dirty data

```
uv run python -c "  
import sqlite3  
from pydantic import ValidationError  
from schemas import TransactionIn  
con = sqlite3.connect('cardguard.db')  
con.row_factory = sqlite3.Row  
ok = bad = 0  
for r in con.execute('SELECT * FROM transactions LIMIT 500'):  
    try:  
        TransactionIn(**{k: r[k] for k in  
            ('card_ref', 'ts', 'amount', 'merchant_catego  
ry', 'city', 'lat', 'lon')})  
        ok += 1
```

WHAT THIS DOES

- Run this in your terminal, from the project folder.
- `uv run` executes inside the project environment — no venv activation.
- `try/except` handles the failure; finally always cleans up.

Replace Hand-Written Validation with Pydantic Models

7

STEP 7 OF 8 · CONT. 2/2

Load real database rows THROUGH the model to catch any dirty data

```
except ValidationError:
    bad += 1
print(f'validated {ok}, rejected {bad}')
```

WHAT THIS DOES

- Run this in your terminal, from the project folder.
- `uv run` executes inside the project environment — no venv activation.
- `try/except` handles the failure; finally always cleans up.

Replace Hand-Written Validation with Pydantic Models

8

STEP 8 OF 8

Note the trade: hand-written validation is explicit but verbose; Pydantic is declarative and reports all errors at once.

Replace Hand-Written Validation with Pydantic Models

Test it

Valid input constructs and normalises; invalid input reports every field error; ScreeningOut serialises to JSON; 500 real rows validate cleanly.

Expose the Screening Engine as a FastAPI Endpoint

LAB 14

The learner wraps the Topic 2 RuleEngine in a FastAPI POST endpoint. The engine is not modified at all — FastAPI is a thin transport layer over logic that already works.

You'll build

A running API with POST /screen, GET /health and automatic OpenAPI docs

Tools & libraries: uv, FastAPI, uvicorn, Pydantic

Expose the Screening Engine as a FastAPI Endpoint

1

STEP 1 OF 8

Add the web dependencies

```
uv add fastapi uvicorn[standard] httpx
```

WHAT THIS DOES

- Run this in your terminal, from the project folder.
- uv resolves, installs and records the dependency in uv.lock.

Expose the Screening Engine as a FastAPI Endpoint

2

STEP 2 OF 8 · CONT. 1/5

Write the application — note how little code the endpoint needs

```
# main.py
import sqlite3
from fastapi import FastAPI, HTTPException
from schemas import TransactionIn, ScreeningOut, RuleHit
from models import Transaction
from cardholder import Cardholder
from rules import AmountDeviationRule, UnusualHourRule,
    HighRiskCategoryRule, VelocityRule
from engine import RuleEngine

app = FastAPI(title="CardGuard Screening API",
    version="1.0.0")
```

WHAT THIS DOES

- Read the signature first: what goes in, what comes out.
- The docstring states the contract the function promises.
- Invalid input fails fast and loudly, never silently.
- try/except handles the failure; finally always cleans up.
- FastAPI turns this typed function into an HTTP endpoint.

Expose the Screening Engine as a FastAPI Endpoint

2

STEP 2 OF 8 · CONT. 2/5

Write the application — note how little code the endpoint needs

```
ENGINE = RuleEngine([AmountDeviationRule(), UnusualHourRule(),
                    HighRiskCategoryRule(), VelocityRule()])

def get_holder(card_ref: str) -> Cardholder:
    con = sqlite3.connect("cardguard.db")
    try:
        row = con.execute("SELECT
            card_ref,name,home_city,typical_amount "
            "FROM cardholders WHERE
            card_ref=?", (card_ref,)).fetchone()
    finally:
        con.close()
    if row is None:
```

WHAT THIS DOES

- Read the signature first: what goes in, what comes out.
- The docstring states the contract the function promises.
- Invalid input fails fast and loudly, never silently.
- try/except handles the failure; finally always cleans up.
- FastAPI turns this typed function into an HTTP endpoint.

Expose the Screening Engine as a FastAPI Endpoint

2

STEP 2 OF 8 · CONT. 3/5

Write the application — note how little code the endpoint needs

```
        raise HTTPException(status_code=404, detail=f"unknown
        card_ref {card_ref}")
    return Cardholder(*row)

def decide(score: float) -> str:
    if score >= 0.80: return "decline"
    if score >= 0.55: return "review"
    return "approve"

@app.get("/health")
def health() -> dict:
    return {"status": "ok"}
```

WHAT THIS DOES

- Read the signature first: what goes in, what comes out.
- The docstring states the contract the function promises.
- Invalid input fails fast and loudly, never silently.
- try/except handles the failure; finally always cleans up.
- FastAPI turns this typed function into an HTTP endpoint.

Expose the Screening Engine as a FastAPI Endpoint

2

STEP 2 OF 8 · CONT. 4/5

Write the application — note how little code the endpoint needs

```
@app.post("/screen", response_model=ScreeningOut)
def screen(txn_in: TransactionIn) -> ScreeningOut:
    """Score one transaction and return the decision with
    reasons."""
    holder = get_holder(txn_in.card_ref)
    txn = Transaction(id=0, card_ref=txn_in.card_ref,
                     ts=txn_in.ts,
                     amount=txn_in.amount,
                     merchant_category=txn_in.merchant_category,
                     city=txn_in.city, lat=txn_in.lat,
                     lon=txn_in.lon)
    result = ENGINE.screen(txn, holder, [])
    return ScreeningOut(card_ref=result.card_ref,
```

WHAT THIS DOES

- Read the signature first: what goes in, what comes out.
- The docstring states the contract the function promises.
- Invalid input fails fast and loudly, never silently.
- try/except handles the failure; finally always cleans up.
- FastAPI turns this typed function into an HTTP endpoint.

Expose the Screening Engine as a FastAPI Endpoint

2

STEP 2 OF 8 · CONT. 5/5

Write the application — note how little code the endpoint needs

```
composite_score=result.composite_score,
                flagged=result.flagged,

decision=decide(result.composite_score),
            hits=[RuleHit(rule_name=r.rule_name,
score=r.score, reason=r.reason)
                for r in result.results if
r.score > 0],
            errors=result.errors)
```

WHAT THIS DOES

- Read the signature first: what goes in, what comes out.
- The docstring states the contract the function promises.
- Invalid input fails fast and loudly, never silently.
- try/except handles the failure; finally always cleans up.
- FastAPI turns this typed function into an HTTP endpoint.

Expose the Screening Engine as a FastAPI Endpoint

3

STEP 3 OF 8

Start the server

```
uv run uvicorn main:app --reload --port 8000
```

WHAT THIS DOES

- Run this in your terminal, from the project folder.
- `uv run` executes inside the project environment — no venv activation.

Expose the Screening Engine as a FastAPI Endpoint

4

STEP 4 OF 8

Open the auto-generated interactive docs — you wrote no OpenAPI spec

```
open http://127.0.0.1:8000/docs
```

WHAT THIS DOES

- Run this in your terminal, from the project folder.

Expose the Screening Engine as a FastAPI Endpoint

5

STEP 5 OF 8

Screen a clearly fraudulent transaction

```
curl -s -X POST http://127.0.0.1:8000/screen -H
"Content-Type: application/json" -d
'{"card_ref":"CH0023","ts":"2026-04-15T02:14:00","amount":
5128.33,"merchant_category":"jewellery","city":"Singapore"
,"lat":1.3521,"lon":103.8198}' | python3 -m json.tool
```

WHAT THIS DOES

- Run this in your terminal, from the project folder.

Expose the Screening Engine as a FastAPI Endpoint

6

STEP 6 OF 8

Screen a normal grocery run and compare the decision

```
curl -s -X POST http://127.0.0.1:8000/screen -H  
"Content-Type: application/json" -d  
'{"card_ref":"CH0001","ts":"2026-04-15T10:30:00","amount":  
86.40,"merchant_category":"grocery","city":"Singapore","la  
t":1.3521,"lon":103.8198}' | python3 -m json.tool
```

WHAT THIS DOES

- Run this in your terminal, from the project folder.

Expose the Screening Engine as a FastAPI Endpoint

7

STEP 7 OF 8

Send invalid input — FastAPI rejects it with 422 before your code runs

```
curl -s -X POST http://127.0.0.1:8000/screen -H  
"Content-Type: application/json" -d  
'{"card_ref":"NOPE","ts":"2026-04-15T10:30:00","amount":-5  
,"merchant_category":"casino","city":"S","lat":1.35,"lon":  
103.8}' | python3 -m json.tool
```

WHAT THIS DOES

- Run this in your terminal, from the project folder.

Expose the Screening Engine as a FastAPI Endpoint

8

STEP 8 OF 8

Send an unknown card and confirm the 404 path

```
curl -s -i -X POST http://127.0.0.1:8000/screen -H  
"Content-Type: application/json" -d  
'{"card_ref":"CH9999","ts":"2026-04-15T10:30:00","amount":  
50,"merchant_category":"grocery","city":"Singapore","lat":  
1.35,"lon":103.8}' | head -1
```

WHAT THIS DOES

- Run this in your terminal, from the project folder.

Expose the Screening Engine as a FastAPI Endpoint

Test it

GET /health returns ok; the fraud transaction returns decline with reasons; the grocery transaction returns approve; bad input returns 422; an unknown card returns 404.

Persist Screening Decisions to SQLite

LAB 15

The learner adds a repository layer that records every screening decision, so the service builds an audit trail — a regulatory requirement for real fraud systems, and the data the Topic 5 dashboard reads.

You'll build

A screenings table plus a repository module with save and query functions

Tools & libraries: uv, sqlite3, FastAPI

Persist Screening Decisions to SQLite

1

STEP 1 OF 7 · CONT. 1/3

Create the audit table

```
# repository.py
import sqlite3
import json
from contextlib import contextmanager
from pathlib import Path

DB = "cardguard.db"

@contextmanager
def connect(db_path: str = DB):
    """Connection that always commits or rolls back, and
    always closes."""
    con = sqlite3.connect(db_path)
```

WHAT THIS DOES

- Read the signature first: what goes in, what comes out.
- The docstring states the contract the function promises.
- try/except handles the failure; finally always cleans up.

Persist Screening Decisions to SQLite

1

STEP 1 OF 7 · CONT. 2/3

Create the audit table

```
con.row_factory = sqlite3.Row
try:
    yield con
    con.commit()
except Exception:
    con.rollback()
    raise
finally:
    con.close()

def init_schema(db_path: str = DB) -> None:
    with connect(db_path) as con:
        con.execute("""
```

WHAT THIS DOES

- Read the signature first: what goes in, what comes out.
- The docstring states the contract the function promises.
- try/except handles the failure; finally always cleans up.

Persist Screening Decisions to SQLite

1

STEP 1 OF 7 · CONT. 3/3

Create the audit table

```
CREATE TABLE IF NOT EXISTS screenings (  
    id INTEGER PRIMARY KEY AUTOINCREMENT,  
    card_ref TEXT NOT NULL,  
    ts TEXT NOT NULL,  
    amount REAL NOT NULL,  
    merchant_category TEXT NOT NULL,  
    composite_score REAL NOT NULL,  
    decision TEXT NOT NULL,  
    hits_json TEXT NOT NULL,  
    screened_at TEXT NOT NULL DEFAULT  
    (datetime('now'))""")  
con.execute("CREATE INDEX IF NOT EXISTS idx_scr_card  
ON screenings(card_ref)")
```

WHAT THIS DOES

- Read the signature first: what goes in, what comes out.
- The docstring states the contract the function promises.
- try/except handles the failure; finally always cleans up.

Persist Screening Decisions to SQLite

2

STEP 2 OF 7 · CONT. 1/3

Add save and query functions — parameterised, never string-formatted

```
# repository.py (add)
def save_screening(txn_in, result, db_path: str = DB) -> int:
    """Record one decision; returns the new row id."""
    with connect(db_path) as con:
        cur = con.execute(
            "INSERT INTO screenings (card_ref, ts, amount,
            merchant_category,
            " composite_score, decision, hits_json) VALUES
            (?, ?, ?, ?, ?, ?, ?)",
            (txn_in.card_ref, txn_in.ts.isoformat(),
            txn_in.amount,
            txn_in.merchant_category,
            result.composite_score, result.decision,
```

WHAT THIS DOES

- Read the signature first: what goes in, what comes out.
- The docstring states the contract the function promises.

Persist Screening Decisions to SQLite

2

STEP 2 OF 7 · CONT. 2/3

Add save and query functions — parameterised, never string-formatted

```
        json.dumps([h.model_dump() for h in
                    result.hits])))
    return cur.lastrowid

def recent_screenings(limit: int = 20, db_path: str = DB) ->
    list[dict]:
    with connect(db_path) as con:
        rows = con.execute(
            "SELECT * FROM screenings ORDER BY id DESC LIMIT
            ?", (limit,)).fetchall()
        return [dict(r) for r in rows]

def decision_counts(db_path: str = DB) -> dict[str, int]:
```

WHAT THIS DOES

- Read the signature first: what goes in, what comes out.
- The docstring states the contract the function promises.

Persist Screening Decisions to SQLite

2

STEP 2 OF 7 · CONT. 3/3

Add save and query functions — parameterised, never string-formatted

```
with connect(db_path) as con:
    rows = con.execute(
        "SELECT decision, COUNT(*) n FROM screenings
        GROUP BY decision").fetchall()
    return {r["decision"]: r["n"] for r in rows}
```

WHAT THIS DOES

- Read the signature first: what goes in, what comes out.
- The docstring states the contract the function promises.

Persist Screening Decisions to SQLite

3

STEP 3 OF 7

Show why parameterised queries matter — the injection an f-string would allow

```
uv run python -c "  
card = \"CH0001\"; DROP TABLE screenings; --\"  
print('UNSAFE would build:')  
print(f\"  SELECT * FROM screenings WHERE card_ref =  
  '{card}'\")  
print('SAFE passes the value separately: con.execute(sql,  
  (card,))')  
"
```

WHAT THIS DOES

- Run this in your terminal, from the project folder.
- `uv run` executes inside the project environment — no `venv` activation.

Persist Screening Decisions to SQLite

4

STEP 4 OF 7 · CONT. 1/2

Wire persistence into the endpoint

```
# main.py (add)
from repository import init_schema, save_screening,
    recent_screenings, decision_counts

@app.on_event("startup")
def _startup() -> None:
    init_schema()

# inside screen(), just before returning:
#     save_screening(txn_in, out)

@app.get("/screenings")
def list_screenings(limit: int = 20) -> list[dict]:
```

WHAT THIS DOES

- Read the signature first: what goes in, what comes out.
- The docstring states the contract the function promises.
- FastAPI turns this typed function into an HTTP endpoint.

Persist Screening Decisions to SQLite

4

STEP 4 OF 7 · CONT. 2/2

Wire persistence into the endpoint

```
"""Recent decisions, newest first – the audit trail."""
return recent_screenings(limit)

@app.get("/stats")
def stats() -> dict:
    """Decision counts for the dashboard."""
    return decision_counts()
```

WHAT THIS DOES

- Read the signature first: what goes in, what comes out.
- The docstring states the contract the function promises.
- FastAPI turns this typed function into an HTTP endpoint.

Persist Screening Decisions to SQLite

5

STEP 5 OF 7

Restart and post several transactions to build history

```
for amt in 86.40 5128.33 240.00 3900.00; do
  curl -s -X POST http://127.0.0.1:8000/screen -H
    "Content-Type: application/json" \
    -d
      '{"card_ref\":"CH0001\","ts\":"2026-04-15T02:3
0:00\","amount\":$amt,"merchant_category\":"ele
ctronics\","city\":"Singapore\","lat\:1.3521,\
lon\:103.8198}' > /dev/null
done
curl -s http://127.0.0.1:8000/stats | python3 -m json.tool
```

WHAT THIS DOES

- Restart and post several transactions to build history

Persist Screening Decisions to SQLite

6

STEP 6 OF 7

Read the audit trail back

```
curl -s 'http://127.0.0.1:8000/screenings?limit=5' | python3  
-m json.tool
```

WHAT THIS DOES

- Run this in your terminal, from the project folder.

Persist Screening Decisions to SQLite

7

STEP 7 OF 7 · CONT. 1/2

Verify rollback works — a failed write must leave no partial row

```
uv run python -c "  
import repository as repo  
repo.init_schema()  
before = len(repo.recent_screenings(1000))  
try:  
    with repo.connect() as con:  
        con.execute('INSERT INTO screenings  
            (card_ref,ts,amount,merchant_category,comp  
             site_score,decision,hits_json) VALUES  
            (?, ?, ?, ?, ?, ?, ?) ',  
  
            ('CH0001', '2026-04-15T10:00:00', 50.0, 'groc  
             ery', 0.1, 'approve', '[]'))
```

WHAT THIS DOES

- Run this in your terminal, from the project folder.
- `uv run` executes inside the project environment — no venv activation.
- Invalid input fails fast and loudly, never silently.
- `try/except` handles the failure; finally always cleans up.

Persist Screening Decisions to SQLite

7

STEP 7 OF 7 · CONT. 2/2

Verify rollback works — a failed write must leave no partial row

```
        raise RuntimeError('simulated failure after insert')
except RuntimeError as e:
    print('caught:', e)
after = len(repo.recent_screenings(1000))
print(f'rows before {before}, after {after} -> rolled back:
      {before == after}')
```

WHAT THIS DOES

- Run this in your terminal, from the project folder.
- `uv run` executes inside the project environment — no venv activation.
- Invalid input fails fast and loudly, never silently.
- `try/except` handles the failure; finally always cleans up.

Persist Screening Decisions to SQLite

Test it

Screenings persist and are queryable via `/screenings` and `/stats`; the injection demo shows why parameters are used; the rollback test proves the row count is unchanged after a mid-transaction failure.

Test the API with pytest and httpx

LAB 16

The learner writes a test suite covering the happy path, validation failures and persistence — so that later refactoring and deployment changes are provably safe.

You'll build

A passing pytest suite against the screening API

Tools & libraries: uv, pytest, FastAPI TestClient

Test the API with pytest and httpx

1

STEP 1 OF 7

Add the test dependencies

```
uv add --dev pytest httpx
```

WHAT THIS DOES

- Run this in your terminal, from the project folder.
- uv resolves, installs and records the dependency in uv.lock.

Test the API with pytest and httpx

2

STEP 2 OF 7 · CONT. 1/3

Write tests for the decision paths

```
# test_api.py
import pytest
from fastapi.testclient import TestClient
from main import app

client = TestClient(app)

def post(**overrides):
    body = {"card_ref": "CH0001", "ts":
        "2026-04-15T10:30:00", "amount": 86.40,
        "merchant_category": "grocery", "city":
        "Singapore",
        "lat": 1.3521, "lon": 103.8198}
```

WHAT THIS DOES

- Read the signature first: what goes in, what comes out.
- The assertion is the test — it must fail before you trust it.

Test the API with pytest and httpx

2

STEP 2 OF 7 · CONT. 2/3

Write tests for the decision paths

```
body.update(overrides)
return client.post("/screen", json=body)

def test_health():
    assert client.get("/health").json() == {"status": "ok"}

def test_normal_transaction_is_approved():
    r = post()
    assert r.status_code == 200
    assert r.json()["decision"] == "approve"

def test_large_offhours_transaction_is_escalated():
    r = post(amount=5128.33, merchant_category="jewellery",
```

WHAT THIS DOES

- Read the signature first: what goes in, what comes out.
- The assertion is the test — it must fail before you trust it.

Test the API with pytest and httpx

2

STEP 2 OF 7 · CONT. 3/3

Write tests for the decision paths

```
ts="2026-04-15T02:14:00")
body = r.json()
assert body["decision"] in {"review", "decline"}
assert body["hits"], "an escalated decision must carry
    its reasons"
```

WHAT THIS DOES

- Read the signature first: what goes in, what comes out.
- The assertion is the test — it must fail before you trust it.

Test the API with pytest and httpx

3

STEP 3 OF 7 · CONT. 1/2

Add tests for the failure paths

```
# test_api.py (add)
@pytest.mark.parametrize("bad", [
    {"amount": -5},
    {"amount": 0},
    {"card_ref": "NOPE"},
    {"merchant_category": "casino"},
    {"lat": 999},
])
def test_invalid_input_is_rejected(bad):
    assert post(**bad).status_code == 422

def test_unknown_card_returns_404():
    assert post(card_ref="CH9999").status_code == 404
```

WHAT THIS DOES

- Read the signature first: what goes in, what comes out.
- The assertion is the test — it must fail before you trust it.

Test the API with pytest and httpx

3

STEP 3 OF 7 · CONT. 2/2

Add tests for the failure paths

```
def test_screening_is_persisted():  
    before = len(client.get("/screenings?limit=1000").json())  
    post()  
    after = len(client.get("/screenings?limit=1000").json())  
    assert after == before + 1
```

WHAT THIS DOES

- Read the signature first: what goes in, what comes out.
- The assertion is the test — it must fail before you trust it.

Test the API with pytest and httpx

4

STEP 4 OF 7

Run the suite

```
uv run pytest -v
```

WHAT THIS DOES

- Run this in your terminal, from the project folder.
- `uv run` executes inside the project environment — no venv activation.

Test the API with pytest and httpx

5

STEP 5 OF 7

Check what the tests actually cover

```
uv add --dev pytest-cov
uv run pytest --cov=. --cov-report=term-missing
```

WHAT THIS DOES

- Run this in your terminal, from the project folder.
- uv resolves, installs and records the dependency in uv.lock.
- uv run executes inside the project environment — no venv activation.

Test the API with pytest and httpx

6

STEP 6 OF 7

Ask your assistant to write a test for a gap — then verify it FAILS first

```
Ask the assistant: Write a pytest test asserting that POST
/screen with amount exactly 1000000 succeeds but 1000001
returns 422, matching the Field(le=1_000_000) constraint
in TransactionIn.
```

WHAT THIS DOES

- Ask your assistant to write a test for a gap — then verify it FAILS first

Test the API with pytest and httpx

7

STEP 7 OF 7

Note the discipline: a test you have never seen fail is a test you cannot trust.

Test the API with pytest and httpx

Test it

All tests pass; invalid inputs are parameterised and rejected with 422; the persistence test proves a row is written per screening.

Recap — Data Modelling with Pydantic and FastAPI

- You can now: Declare data shape with type hints and validate at runtime
- You can now: Turn typed Python functions into HTTP endpoints
- You can now: Write application state to a database from the API layer
- You can now: Write automated tests for a FastAPI application

TOPIC 05

Packaging and Deployment

uv Lockfiles · Configuration · Containers · Deploying a Service

Key Concepts — Packaging and Deployment

1

uv.lock pins exact versions so the application installs identically in development and production.

2

Configuration belongs in environment variables, never hard-coded — secrets must never be committed.

3

A container image bundles the interpreter, dependencies and application code into one deployable artifact.

4

A deployment is not done until the running service has been verified with a real request against a health endpoint.

Hands-On Labs — Packaging and Deployment

Labs 17–18

- Build a Fraud Analyst Dashboard with Streamlit
- Externalise Configuration and Protect Secrets

Labs 19–20

- Lock Dependencies and Containerise the API
- Deploy the Full Stack with Docker Compose

—

- —

Build a Fraud Analyst Dashboard with Streamlit

LAB 17

The learner builds the third tier: a Streamlit dashboard a fraud analyst actually uses — review queue, screening form and trend charts — calling the FastAPI service rather than reaching into the database.

You'll build

A running Streamlit dashboard with live screening, a review queue and charts

Tools & libraries: uv, Streamlit, FastAPI, httpx, pandas

Build a Fraud Analyst Dashboard with Streamlit

1

STEP 1 OF 8

Add Streamlit

```
uv add streamlit httpx
```

WHAT THIS DOES

- Run this in your terminal, from the project folder.
- uv resolves, installs and records the dependency in uv.lock.

Build a Fraud Analyst Dashboard with Streamlit

2

STEP 2 OF 8 · CONT. 1/5

Build the dashboard shell and the live screening form

```
# app.py
import httpx
import pandas as pd
import streamlit as st

API = st.secrets.get("api_url", "http://127.0.0.1:8000")

st.set_page_config(page_title="CardGuard",
                    page_icon="\U0001F6E1", layout="wide")
st.title("CardGuard – Fraud Screening Console")

with st.sidebar:
    st.header("Screen a transaction")
```

WHAT THIS DOES

- try/except handles the failure; finally always cleans up.

Build a Fraud Analyst Dashboard with Streamlit

2

STEP 2 OF 8 · CONT. 2/5

Build the dashboard shell and the live screening form

```
card_ref = st.text_input("Card reference", "CH0001")
amount = st.number_input("Amount (SGD)", min_value=0.01,
    value=86.40, step=10.0)
category = st.selectbox("Merchant category",

    ["grocery", "fuel", "restaurant", "electronic
    s", "online_gaming",

    "jewellery", "crypto_exchange", "gift_cards"
    , "pharmacy", "transport"])
hour = st.slider("Hour of day", 0, 23, 10)
submitted = st.button("Screen", type="primary")
```

WHAT THIS DOES

- try/except handles the failure; finally always cleans up.

Build a Fraud Analyst Dashboard with Streamlit

2

STEP 2 OF 8 · CONT. 3/5

Build the dashboard shell and the live screening form

```
if submitted:
    payload = {"card_ref": card_ref, "ts":
        f"2026-04-15T{hour:02d}:30:00",
        "amount": float(amount), "merchant_category":
            category,
            "city": "Singapore", "lat": 1.3521, "lon":
                103.8198}
    try:
        r = httpx.post(f"{API}/screen", json=payload,
            timeout=10)
        r.raise_for_status()
        out = r.json()
    except httpx.HTTPStatusError as exc:
```

WHAT THIS DOES

- try/except handles the failure; finally always cleans up.

Build a Fraud Analyst Dashboard with Streamlit

2

STEP 2 OF 8 · CONT. 4/5

Build the dashboard shell and the live screening form

```
        st.error(f"API rejected the request\n        ({exc.response.status_code}): {exc.response.text}")\n    except httpx.RequestError:\n        st.error("Cannot reach the screening API. Is it\n        running on port 8000?")\n    else:\n        colour = {"approve": "green", "review": "orange",\n                  "decline": "red"}[out["decision"]]\n        st.markdown(f"### Decision:\n        :{colour}[{out['decision'].upper()}]")\n        st.metric("Composite score", out["composite_score"])\n        if out["hits"]:\n            st.dataframe(pd.DataFrame(out["hits"]),
```

WHAT THIS DOES

- try/except handles the failure; finally always cleans up.

Build a Fraud Analyst Dashboard with Streamlit

2

STEP 2 OF 8 · CONT. 5/5

Build the dashboard shell and the live screening form

```
width="stretch")
else:
    st.info("No rule fired on this transaction.")
```

WHAT THIS DOES

- try/except handles the failure; finally always cleans up.

Build a Fraud Analyst Dashboard with Streamlit

3

STEP 3 OF 8 · CONT. 1/3

Add the review queue and trend charts

```
# app.py (add)
st.divider()
left, right = st.columns([2, 1])

try:
    rows = httpx.get(f"{API}/screenings", params={"limit":
        200}, timeout=10).json()
    stats = httpx.get(f"{API}/stats", timeout=10).json()
except httpx.RequestError:
    st.warning("API unavailable – showing no history.")
    rows, stats = [], {}

with left:
```

WHAT THIS DOES

- try/except handles the failure; finally always cleans up.

Build a Fraud Analyst Dashboard with Streamlit

3

STEP 3 OF 8 · CONT. 2/3

Add the review queue and trend charts

```
st.subheader("Recent screenings")
if rows:
    df = pd.DataFrame(rows)
    df["screened_at"] = pd.to_datetime(df["screened_at"])
    st.dataframe(

        df[["card_ref", "amount", "merchant_category",
            "composite_score", "decision", "screened_at"]],
        width="stretch", height=340)
else:
    st.info("No screenings yet – submit one from the sidebar.")
```

WHAT THIS DOES

- try/except handles the failure; finally always cleans up.

Build a Fraud Analyst Dashboard with Streamlit

3

STEP 3 OF 8 · CONT. 3/3

Add the review queue and trend charts

```
with right:
    st.subheader("Decisions")
    if stats:
        st.bar_chart(pd.Series(stats, name="count"))
        total = sum(stats.values())
        flagged = stats.get("review", 0) +
            stats.get("decline", 0)
        st.metric("Flag rate", f"{100 * flagged /
            total:.1f}%" if total else "-")
```

WHAT THIS DOES

- try/except handles the failure; finally always cleans up.

Build a Fraud Analyst Dashboard with Streamlit

4

STEP 4 OF 8

Run both tiers — API in one terminal, UI in another

```
# terminal 1
uv run uvicorn main:app --reload --port 8000

# terminal 2
uv run streamlit run app.py
```

WHAT THIS DOES

- `uv run` executes inside the project environment — no `venv` activation.

Build a Fraud Analyst Dashboard with Streamlit

5

STEP 5 OF 8

Screen a normal transaction in the UI and watch it land in the queue

```
open http://localhost:8501
```

WHAT THIS DOES

- Run this in your terminal, from the project folder.

Build a Fraud Analyst Dashboard with Streamlit

6

STEP 6 OF 8

Screen a \$5,000 jewellery charge at 02:00 and confirm it turns red

Build a Fraud Analyst Dashboard with Streamlit

7

STEP 7 OF 8

Stop the API and confirm the UI degrades gracefully instead of crashing

```
# stop uvicorn (Ctrl-C), then click Screen in the UI
```

WHAT THIS DOES

- Stop the API and confirm the UI degrades gracefully instead of crashing

Build a Fraud Analyst Dashboard with Streamlit

8

STEP 8 OF 8

Note the architecture: the UI never touches SQLite. Swapping the database would not change app.py at all.

Build a Fraud Analyst Dashboard with Streamlit

Test it

The dashboard screens transactions live, colour-codes decisions, lists the review queue and charts decision counts; with the API stopped it shows a clear error rather than a traceback.

Externalise Configuration and Protect Secrets

LAB 18

The learner replaces every hard-coded value with typed settings loaded from the environment, and ensures secrets can never be committed — the change that makes the same image runnable in dev and production.

You'll build

A typed Settings object, a .env file and a .gitignore that excludes it

Tools & libraries: uv, pydantic-settings, python-dotenv

Externalise Configuration and Protect Secrets

1

STEP 1 OF 9

Find every hard-coded value in the project

```
grep -rn '127.0.0.1\|8000\|cardguard.db\|0.55\|0.80'  
--include='*.py' . | grep -v '.venv'
```

WHAT THIS DOES

- Run this in your terminal, from the project folder.

Externalise Configuration and Protect Secrets

2

STEP 2 OF 9

Add typed settings

```
uv add pydantic-settings
```

WHAT THIS DOES

- Run this in your terminal, from the project folder.
- uv resolves, installs and records the dependency in uv.lock.

Externalise Configuration and Protect Secrets

3

STEP 3 OF 9 · CONT. 1/2

Declare configuration as a validated model

```
# config.py
from pydantic_settings import BaseSettings, SettingsConfigDict

class Settings(BaseSettings):
    """All runtime configuration, validated at startup."""
    model_config = SettingsConfigDict(env_file=".env",
                                       env_prefix="CARDGUARD_")

    db_path: str = "cardguard.db"
    api_url: str = "http://127.0.0.1:8000"
    review_threshold: float = 0.55
    decline_threshold: float = 0.80
    velocity_window_minutes: int = 10
```

WHAT THIS DOES

- The docstring states the contract the function promises.
- The class bundles data with the behaviour that acts on it.

Externalise Configuration and Protect Secrets

3

STEP 3 OF 9 · CONT. 2/2

Declare configuration as a validated model

```
log_level: str = "INFO"  
settings = Settings()
```

WHAT THIS DOES

- The docstring states the contract the function promises.
- The class bundles data with the behaviour that acts on it.

Externalise Configuration and Protect Secrets

4

STEP 4 OF 9

Create .env for local development and EXCLUDE it from git immediately

```
# .env
CARDGUARD_DB_PATH=cardguard.db
CARDGUARD_REVIEW_THRESHOLD=0.55
CARDGUARD_DECLINE_THRESHOLD=0.80
CARDGUARD_LOG_LEVEL=DEBUG
```

WHAT THIS DOES

- Create .env for local development and EXCLUDE it from git immediately

Externalise Configuration and Protect Secrets

5

STEP 5 OF 9

Add the gitignore entries BEFORE the first commit

```
# .gitignore
.env
.env.*
!.env.example
.venv/
__pycache__/
*.db
.pytest_cache/
```

WHAT THIS DOES

- Add the gitignore entries BEFORE the first commit

Externalise Configuration and Protect Secrets

6

STEP 6 OF 9

Commit a `.env.example` instead, so a new joiner knows what to set

```
# .env.example – safe to commit, contains no real values
CARDGUARD_DB_PATH=cardguard.db
CARDGUARD_REVIEW_THRESHOLD=0.55
CARDGUARD_DECLINE_THRESHOLD=0.80
CARDGUARD_LOG_LEVEL=INFO
```

WHAT THIS DOES

- Commit a `.env.example` instead, so a new joiner knows what to set

Externalise Configuration and Protect Secrets

7

STEP 7 OF 9

Replace the hard-coded thresholds with settings

```
# main.py (replace decide())
from config import settings

def decide(score: float) -> str:
    if score >= settings.decline_threshold: return "decline"
    if score >= settings.review_threshold: return "review"
    return "approve"
```

WHAT THIS DOES

- Read the signature first: what goes in, what comes out.

Externalise Configuration and Protect Secrets

8

STEP 8 OF 9

Prove configuration is live — override without editing any code

```
uv run python -c "  
from config import settings  
print('default review threshold:', settings.review_threshold)  
"  
CARDGUARD_REVIEW_THRESHOLD=0.30 uv run python -c "  
from config import settings  
print('overridden by environment:', settings.review_threshold)  
"
```

WHAT THIS DOES

- Run this in your terminal, from the project folder.
- `uv run` executes inside the project environment — no venv activation.

Externalise Configuration and Protect Secrets

9

STEP 9 OF 9

Confirm .env is genuinely ignored

```
git check-ignore -v .env && echo 'SAFE: .env will not be  
committed'
```

WHAT THIS DOES

- Run this in your terminal, from the project folder.

Externalise Configuration and Protect Secrets

Test it

Settings load from `.env`; an environment variable overrides them without a code change; `git check-ignore` confirms `.env` is excluded; `.env.example` is committed in its place.

Lock Dependencies and Containerise the API

LAB 19

The learner writes a multi-stage Dockerfile that installs from uv.lock, runs as a non-root user, and produces an image that behaves identically on any machine.

You'll build

A built Docker image running the screening API with a health check

Tools & libraries: uv, Docker, uvicorn

Lock Dependencies and Containerise the API

1

STEP 1 OF 9

Confirm the lockfile is current and committed

```
uv lock --check  
git add uv.lock pyproject.toml
```

WHAT THIS DOES

- Run this in your terminal, from the project folder.

Lock Dependencies and Containerise the API

2

STEP 2 OF 9 · CONT. 1/2

Write the Dockerfile — multi-stage keeps the final image small

```
# Dockerfile
FROM ghcr.io/astral-sh/uv:python3.12-bookworm-slim AS builder
WORKDIR /app
ENV UV_COMPILE_BYTECODE=1 UV_LINK_MODE=copy

# Install dependencies first so this layer caches across code
# changes
COPY pyproject.toml uv.lock ./
RUN uv sync --frozen --no-install-project --no-dev

COPY . .
RUN uv sync --frozen --no-dev
```

WHAT THIS DOES

- The image bundles interpreter, dependencies and code as one artifact.

Lock Dependencies and Containerise the API

2

STEP 2 OF 9 · CONT. 2/2

Write the Dockerfile — multi-stage keeps the final image small

```
FROM python:3.12-slim-bookworm AS runtime
RUN useradd --create-home --uid 1000 appuser
WORKDIR /app
COPY --from=builder --chown=appuser:appuser /app /app
ENV PATH="/app/.venv/bin:$PATH" PYTHONUNBUFFERED=1
USER appuser
EXPOSE 8000
HEALTHCHECK --interval=30s --timeout=3s --retries=3 \
  CMD python -c "import httpx,sys; sys.exit(0 if
    httpx.get('http://127.0.0.1:8000/health').status_code=
    =200 else 1)"
CMD ["uvicorn", "main:app", "--host", "0.0.0.0", "--port",
    "8000"]
```

WHAT THIS DOES

- The image bundles interpreter, dependencies and code as one artifact.

Lock Dependencies and Containerise the API

3

STEP 3 OF 9

Exclude everything the image does not need

```
# .dockerignore
.venv/
.git/
.env
*.db
__pycache__/
.pytest_cache/
tests/
*.md
```

WHAT THIS DOES

- The image bundles interpreter, dependencies and code as one artifact.

Lock Dependencies and Containerise the API

4

STEP 4 OF 9

Build the image

```
docker build -t cardguard-api:1.0 .
```

WHAT THIS DOES

- Run this in your terminal, from the project folder.
- The image bundles interpreter, dependencies and code as one artifact.

Lock Dependencies and Containerise the API

5

STEP 5 OF 9

Check the size — multi-stage should keep it well under 300MB

```
docker images cardguard-api:1.0
```

WHAT THIS DOES

- Run this in your terminal, from the project folder.
- The image bundles interpreter, dependencies and code as one artifact.

Lock Dependencies and Containerise the API

6

STEP 6 OF 9

Run it, passing configuration as environment variables

```
docker run -d --name cardguard \  
-p 8000:8000 \  
-e CARDGUARD_REVIEW_THRESHOLD=0.55 \  
-e CARDGUARD_LOG_LEVEL=INFO \  
-v "$(pwd)/cardguard.db:/app/cardguard.db" \  
cardguard-api:1.0
```

WHAT THIS DOES

- Run this in your terminal, from the project folder.
- The image bundles interpreter, dependencies and code as one artifact.

Lock Dependencies and Containerise the API

7

STEP 7 OF 9

Verify the container is healthy and answering

```
docker ps --filter name=cardguard --format 'table
{{.Names}}\t{{.Status}}'
curl -s http://127.0.0.1:8000/health
```

WHAT THIS DOES

- Run this in your terminal, from the project folder.
- The image bundles interpreter, dependencies and code as one artifact.

Lock Dependencies and Containerise the API

8

STEP 8 OF 9

Screen a transaction against the CONTAINER, not your laptop's Python

```
curl -s -X POST http://127.0.0.1:8000/screen -H
"Content-Type: application/json" -d
'{"card_ref":"CH0023","ts":"2026-04-15T02:14:00","amount":
5128.33,"merchant_category":"jewellery","city":"Singapore"
,"lat":1.3521,"lon":103.8198}' | python3 -m json.tool
```

WHAT THIS DOES

- Run this in your terminal, from the project folder.

Lock Dependencies and Containerise the API

9

STEP 9 OF 9

Read the logs and confirm it runs as a non-root user

```
docker logs cardguard --tail 20  
docker exec cardguard whoami
```

WHAT THIS DOES

- Run this in your terminal, from the project folder.
- The image bundles interpreter, dependencies and code as one artifact.

Lock Dependencies and Containerise the API

Test it

The image builds, runs as appuser, reports healthy, and returns a decline decision for the fraud transaction posted to the container.

Deploy the Full Stack with Docker Compose

LAB 20

The learner composes the API and the Streamlit UI into one stack with a shared volume and dependency ordering, then verifies the deployment end to end — the final integration of everything built in Topics 1-5.

You'll build

A running two-service stack verified with a real screening through the UI

Tools & libraries: Docker Compose, FastAPI, Streamlit, SQLite

Deploy the Full Stack with Docker Compose

1

STEP 1 OF 10

Write a Dockerfile for the UI tier

```
# Dockerfile.ui
FROM ghcr.io/astral-sh/uv:python3.12-bookworm-slim
WORKDIR /app
ENV UV_COMPILE_BYTECODE=1 UV_LINK_MODE=copy
COPY pyproject.toml uv.lock ./
RUN uv sync --frozen --no-dev
COPY . .
ENV PATH="/app/.venv/bin:$PATH"
EXPOSE 8501
CMD ["streamlit", "run", "app.py",
    "--server.address=0.0.0.0", "--server.port=8501"]
```

WHAT THIS DOES

- The image bundles interpreter, dependencies and code as one artifact.

Deploy the Full Stack with Docker Compose

2

STEP 2 OF 10 · CONT. 1/3

Compose the stack

```
# compose.yaml
services:
  api:
    build: .
    ports: ["8000:8000"]
    environment:
      CARDGUARD_DB_PATH: /data/cardguard.db
      CARDGUARD_REVIEW_THRESHOLD: "0.55"
      CARDGUARD_DECLINE_THRESHOLD: "0.80"
    volumes: ["cardguard-data:/data"]
    healthcheck:
      test: ["CMD", "python", "-c", "import httpx,sys; sys.exit(0 if
```

WHAT THIS DOES

- The image bundles interpreter, dependencies and code as one artifact.

Deploy the Full Stack with Docker Compose

2

STEP 2 OF 10 · CONT. 2/3

Compose the stack

```
        httpx.get('http://127.0.0.1:8000/health').stat
        us_code==200 else 1)"]
interval: 10s
timeout: 3s
retries: 5

ui:
  build:
    context: .
    dockerfile: Dockerfile.ui
  ports: ["8501:8501"]
  environment:
    CARDGUARD_API_URL: http://api:8000
```

WHAT THIS DOES

- The image bundles interpreter, dependencies and code as one artifact.

Deploy the Full Stack with Docker Compose

2

STEP 2 OF 10 · CONT. 3/3

Compose the stack

```
depends_on:
  api:
    condition: service_healthy

volumes:
  cardguard-data:
```

WHAT THIS DOES

- The image bundles interpreter, dependencies and code as one artifact.

Deploy the Full Stack with Docker Compose

3

STEP 3 OF 10

Point the UI at the service name — inside the network it is not localhost

```
# app.py (replace the API constant)
import os
API = os.getenv("CARDGUARD_API_URL", "http://127.0.0.1:8000")
```

WHAT THIS DOES

- Point the UI at the service name — inside the network it is not localhost

Deploy the Full Stack with Docker Compose

4

STEP 4 OF 10

Bring the stack up

```
docker compose up --build -d
```

WHAT THIS DOES

- Run this in your terminal, from the project folder.
- The image bundles interpreter, dependencies and code as one artifact.

Deploy the Full Stack with Docker Compose

5

STEP 5 OF 10

Confirm the UI waited for the API to become healthy

```
docker compose ps  
docker compose logs api --tail 5
```

WHAT THIS DOES

- Run this in your terminal, from the project folder.
- The image bundles interpreter, dependencies and code as one artifact.

Deploy the Full Stack with Docker Compose

6

STEP 6 OF 10

Seed the database inside the running container

```
docker compose exec api python mockdata.py
docker compose exec api ls -la /data
```

WHAT THIS DOES

- Run this in your terminal, from the project folder.
- The image bundles interpreter, dependencies and code as one artifact.

Deploy the Full Stack with Docker Compose

7

STEP 7 OF 10

Verify end to end through the browser — screen a fraudulent charge in the UI

```
open http://localhost:8501
```

WHAT THIS DOES

- Run this in your terminal, from the project folder.

Deploy the Full Stack with Docker Compose

8

STEP 8 OF 10

Verify the API tier independently

```
curl -s http://127.0.0.1:8000/health  
curl -s http://127.0.0.1:8000/stats | python3 -m json.tool
```

WHAT THIS DOES

- Run this in your terminal, from the project folder.

Deploy the Full Stack with Docker Compose

9

STEP 9 OF 10

Confirm data survives a restart — this is what the volume is for

```
docker compose restart api  
sleep 5  
curl -s http://127.0.0.1:8000/stats | python3 -m json.tool
```

WHAT THIS DOES

- Run this in your terminal, from the project folder.
- The image bundles interpreter, dependencies and code as one artifact.

Deploy the Full Stack with Docker Compose

10

STEP 10 OF 10

Tear down, keeping the data volume

```
docker compose down  
docker volume ls | grep cardguard
```

WHAT THIS DOES

- Run this in your terminal, from the project folder.
- The image bundles interpreter, dependencies and code as one artifact.

Deploy the Full Stack with Docker Compose

Test it

Both services start, the UI waits for the API health check, a screening submitted in the browser is stored and visible via /stats, and the data survives a container restart.

Recap — Packaging and Deployment

- You can now: Create a data application UI that consumes the API
- You can now: Move settings out of code into the environment
- You can now: Package the application into a reproducible image
- You can now: Run and verify a multi-service application



WRAP-UP

Course Summary & Next Steps

What You Achieved

1

Vibe Coding Foundations

uv Projects · AI Coding Assistants · Prompting Patterns · AI-Assisted Refactoring

2

Object-Oriented Programming in Python

Classes · Encapsulation · Inheritance · Composition · Dunder Methods

3

Data Analytics with pandas

DataFrames · Cleaning · Transformation · GroupBy · Aggregation · Reporting

4

Data Modelling with Pydantic and FastAPI

Typed Models · Validation · Endpoints · Request/Response Schemas · Auto Docs

5

Packaging and Deployment

uv Lockfiles · Configuration · Containers · Deploying a Service

Where to Go Next

- Rebuild CardGuard from scratch on your own machine, using your AI assistant for every step.
- Add a new fraud rule and a matching pytest test, then redeploy the container.
- Swap SQLite for PostgreSQL — the repository layer is the only module that should change.
- Publish your image to a registry and deploy it to a cloud host.
- Practise the four-part prompt pattern on your own work: goal, constraints, inputs, expected output.

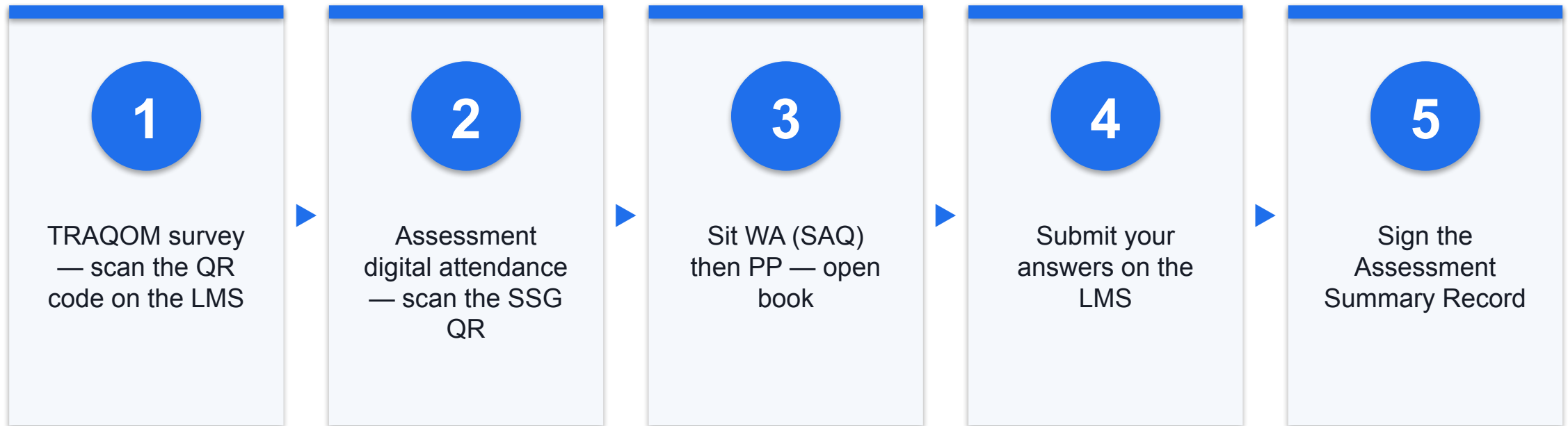
Keep Practising

- Re-run every lab from a clean folder — `uv sync` restores the exact environment from `uv.lock`.
- Read the Learner Guide alongside your own code; the step numbering matches the labs exactly.
- Practise the four-part prompt pattern: goal, constraints, inputs, expected output.
- Tertiary Infotech practice exams: <https://exams.tertiaryinfotech.com/>

Assessment

- Written Assessment (SAQ) — 1 hour. Practical Performance (PP) — 1 hour.
- Open book: slides, Learner Guide and approved materials only.
- Remember to take the Assessment digital attendance (TRAQOM).
- Submit your completed answers on the LMS at <https://lms-tms.tertiaryinfotech.com/>.

Assessment Flow



Digital Attendance (Mandatory)

- It is mandatory to take the AM, PM and Assessment digital attendance for WSQ-funded courses.
- The trainer/administrator displays the digital attendance QR code from the SSG portal.
- Scan the QR code with your mobile phone camera and submit your attendance.
- A minimum of 75% attendance is required to be eligible for assessment and funding.

KEEP BUILDING

Thank You!

You can now build, test and deploy a real Python application with AI assistance.